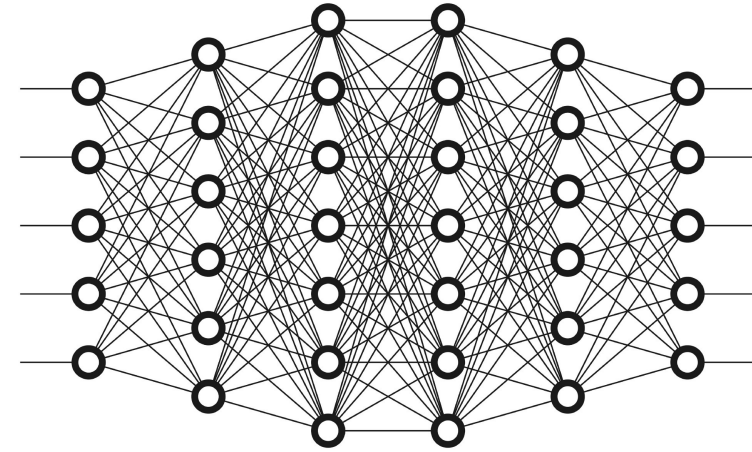


COMP9517

Computer Vision

2026 Term 1 Week 7

Dr Sonit Singh



Deep Learning

Overview of Convolutional Neural
Networks

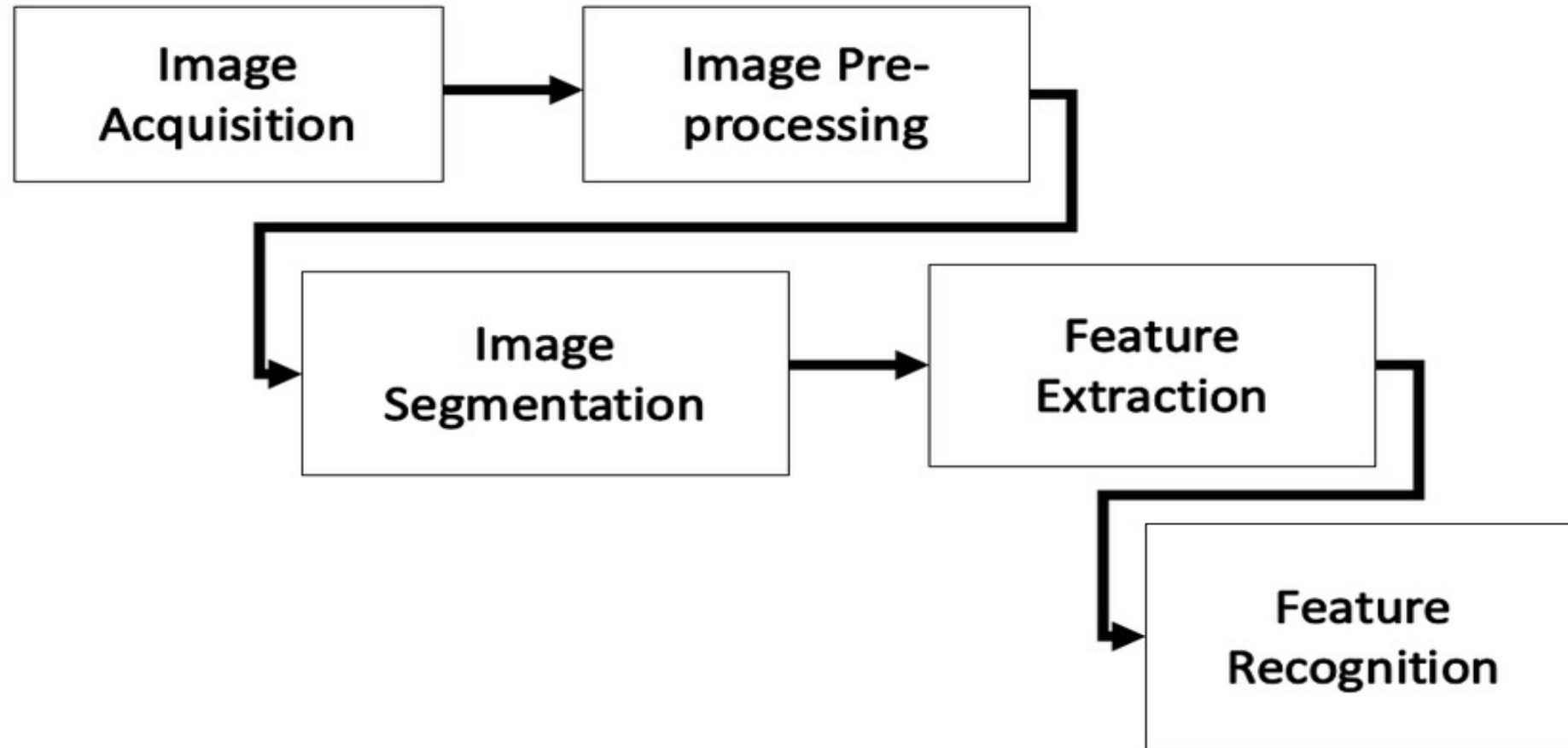
Challenges in CV

Consider object detection as an example:

- Variations in viewpoint
- Differences in illumination
- Hidden parts of images
- Background clutter

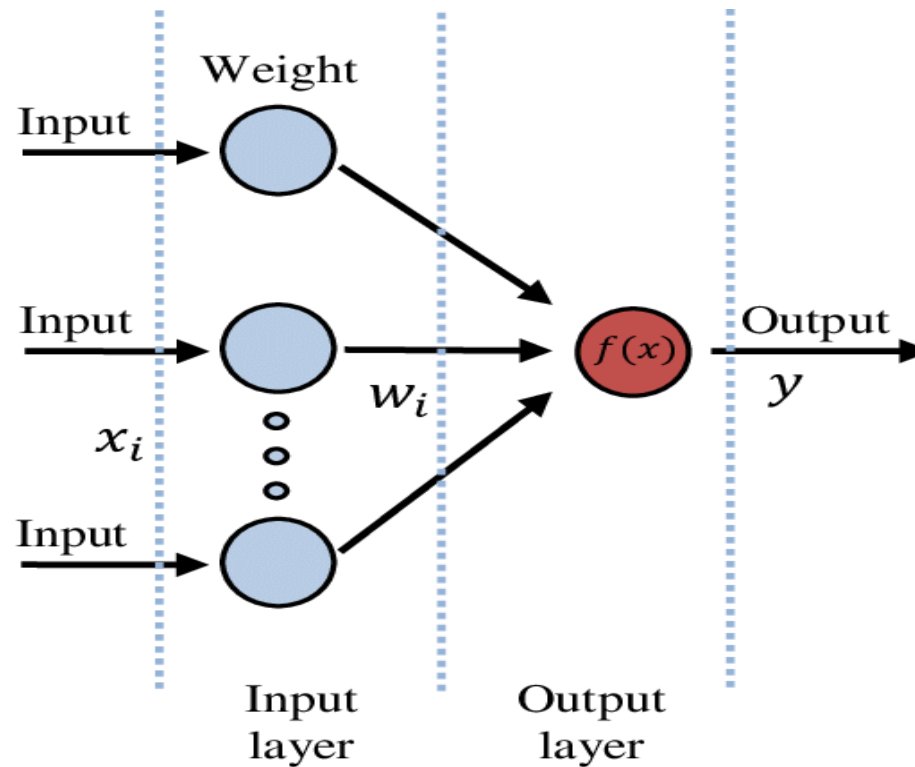


Typical CV pipeline



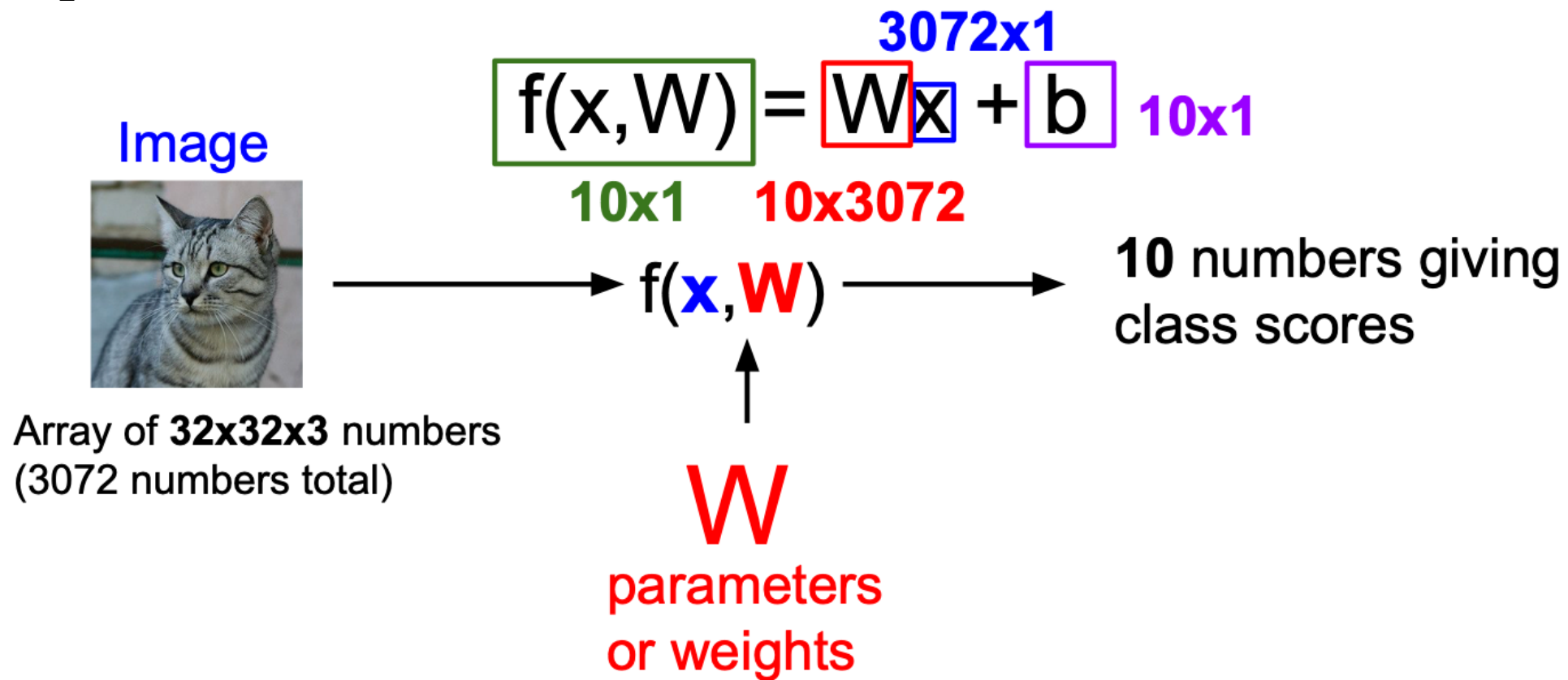
Linear Classifier for Image Classification

A single-layer Neural Network



Linear Classifiers

- Image classification with linear classifier

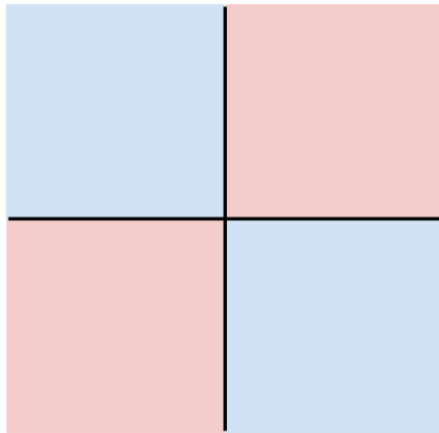


Linear Classifiers

- Hard cases for a linear classifier.
- Extracting better features (manually) may help but cannot (always) solve the problems.

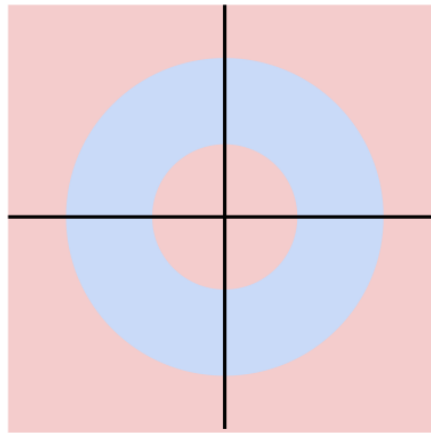
Class 1:
First and third quadrants

Class 2:
Second and fourth quadrants



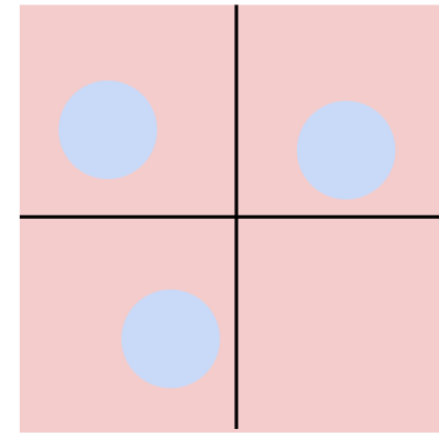
Class 1:
 $1 \leq \text{L2 norm} \leq 2$

Class 2:
Everything else



Class 1:
Three modes

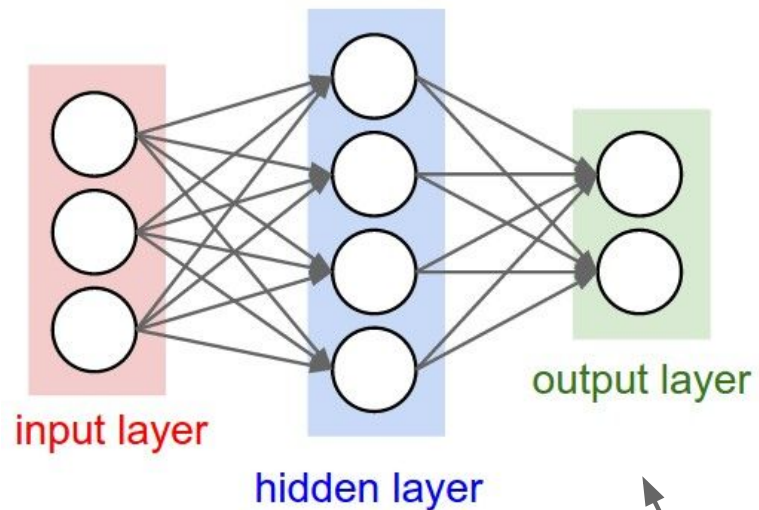
Class 2:
Everything else



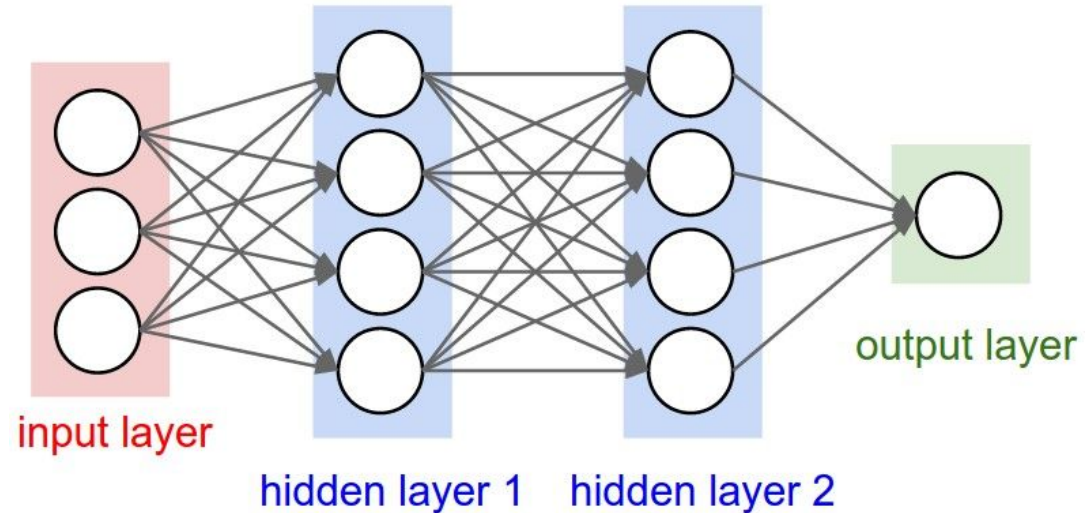
From Linear Classifiers to (Non-linear) Neural Networks

Neural Networks

Architectures (for MLP)



“2-layer Neural Net”, or
“1-hidden-layer Neural Net”



“3-layer Neural Net”, or
“2-hidden-layer Neural Net”

“Fully-connected” layers

Neural Networks

- Starting from the original linear classifier

(Before) Linear score function: $f = Wx$

$$x \in \mathbb{R}^D, W \in \mathbb{R}^{C \times D}$$

Neural Networks

- 2 layers

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

(In practice we will usually add a learnable bias at each layer as well)

Neural Networks

- 2 layers
- Also called as **fully connected network**
- **Fully connected (FC) layer**

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

“Neural Network” is a very broad term; these are more accurately called “fully-connected networks” or sometimes “multi-layer perceptrons” (MLP)

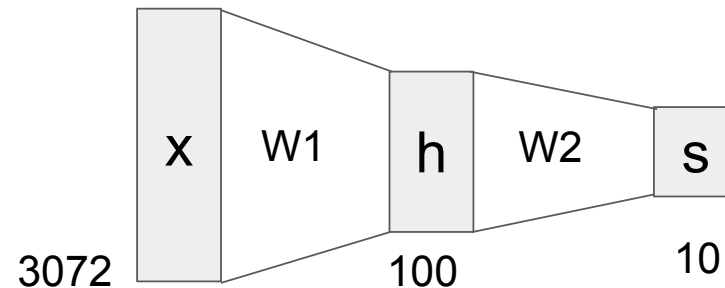
(In practice we will usually add a learnable bias at each layer as well)

Neural Networks

- 2 layers
- Also called as **fully connected network**
- **Fully connected (FC) layer**

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Neural Networks

- 3 layers

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$
or 3-layer Neural Network

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H_1 \times D}, W_2 \in \mathbb{R}^{H_2 \times H_1}, W_3 \in \mathbb{R}^{C \times H_2}$$

(In practice we will usually add a learnable bias at each layer as well)

Neural Networks

- Activation function
- The function $\max(0, z)$ is called the **activation function**.

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

- What if without the activation function?

Neural Networks

Activation function

The function $\max(0, z)$ is called the **activation function**.

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

What if without the activation function?

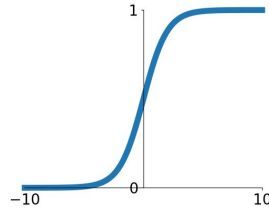
- The model will be linear.

Neural Networks

- Activation functions
 - Non-linear functions

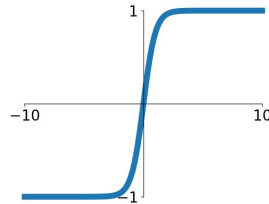
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



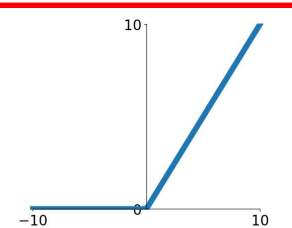
tanh

$$\tanh(x)$$



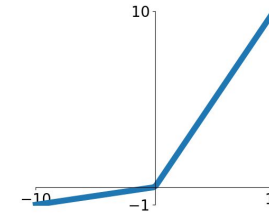
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

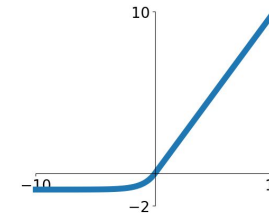


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

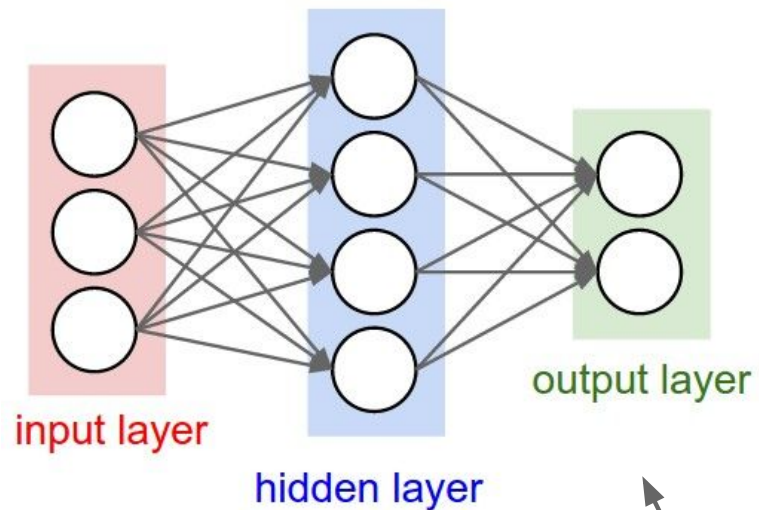
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

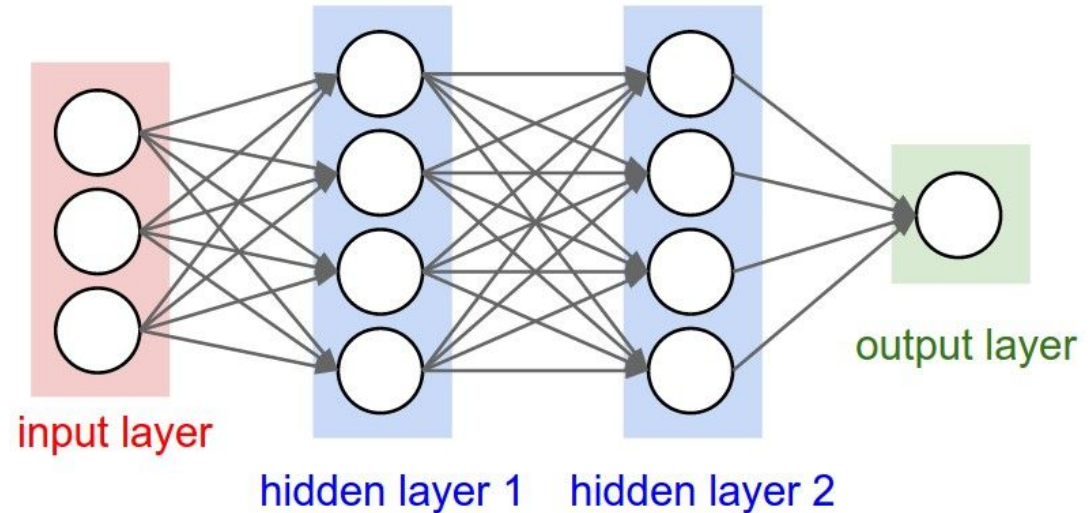


Neural Networks

Architectures (for MLP)



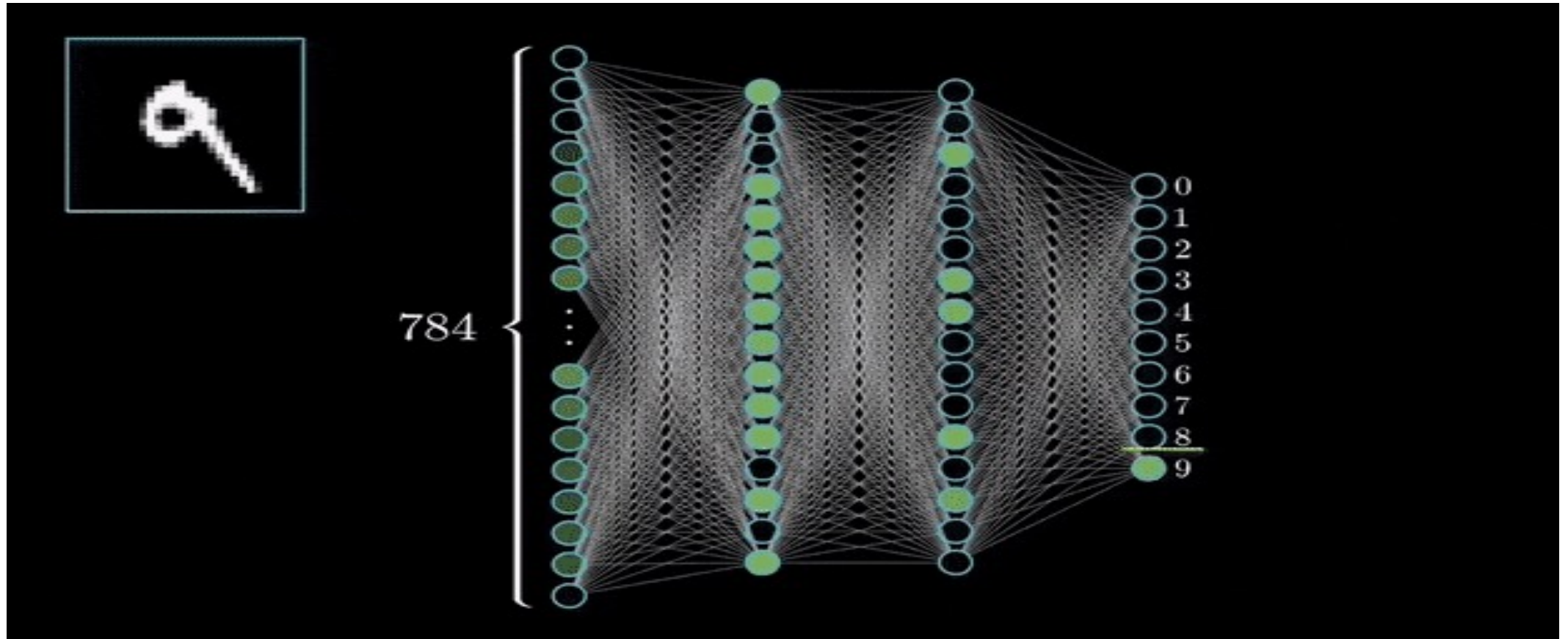
“2-layer Neural Net”, or
“1-hidden-layer Neural Net”



“3-layer Neural Net”, or
“2-hidden-layer Neural Net”

“Fully-connected” layers

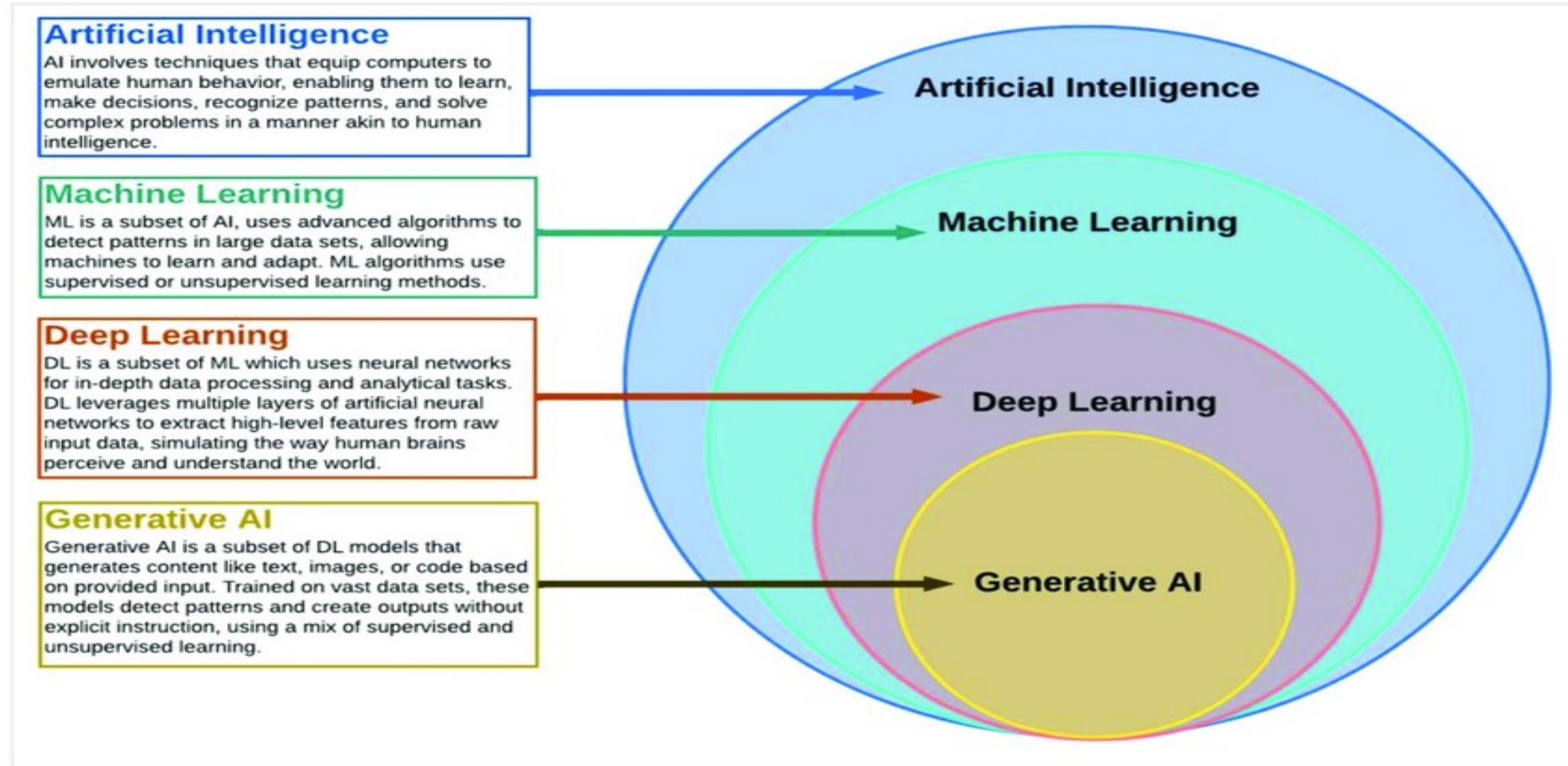
Neural Networks



Source: 3Blue1Brown

From Neural Networks to “Deep Neural Networks” also called “Deep Learning”

What is Deep Learning



Source: The Application of AutoML Techniques in Diabetes Diagnosis: Current Approaches, Performance, and Future Directions

What is Deep Learning

- Deep learning is a subset of machine learning that uses deep neural networks, called deep neural networks, to simulate the complex decision-making power of the human brain.
- Predominantly, deep learning techniques rely on large amounts of data and deeper learning architectures
- Some well-known paradigms for different types of data and applications:
 - **Convolutional Neural Networks (CNNs) (Week 7)**
 - Recurrent Neural Networks
 - Generative Adversarial Networks
 - Transformers

From Neural Networks to “Deep Learning”

First strong results

Acoustic Modeling using Deep Belief Networks

Abdel-rahman Mohamed, George Dahl, Geoffrey Hinton, 2010

Context-Dependent Pre-trained Deep Neural Networks for Large Vocabulary Speech Recognition

George Dahl, Dong Yu, Li Deng, Alex Acero, 2012

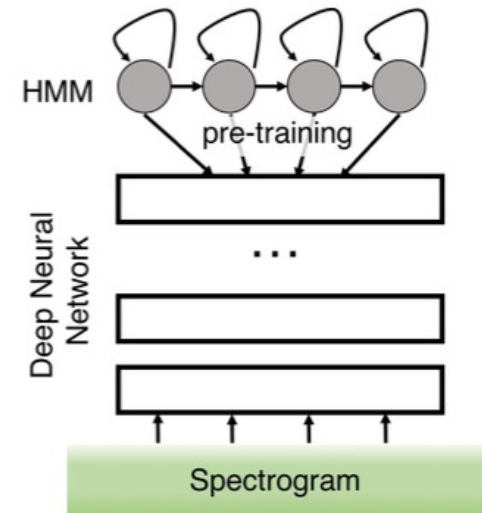
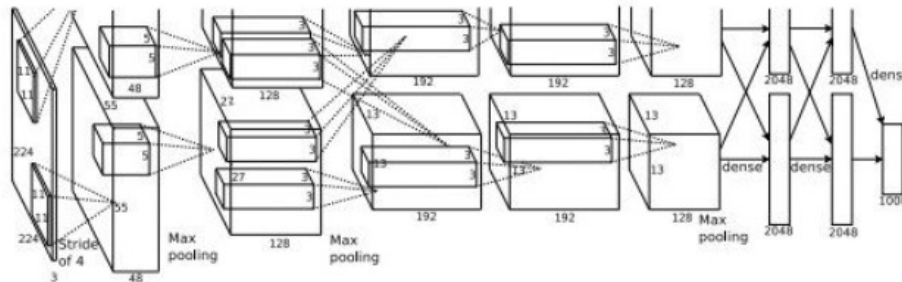


Illustration of Dahl et al. 2012 by Lane McIntosh, copyright CS231n 2017

Imagenet classification with deep convolutional neural networks

Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton, 2012



Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

From Neural Networks to “Deep Learning”

First strong results

Acoustic Modeling using Deep Belief Networks

Abdel-rahman Mohamed, George Dahl, Geoffrey Hinton, 2010

Context-Dependent Pre-trained Deep Neural Networks for Large Vocabulary Speech Recognition

George Dahl, Dong Yu, Li Deng, Alex Acero, 2012

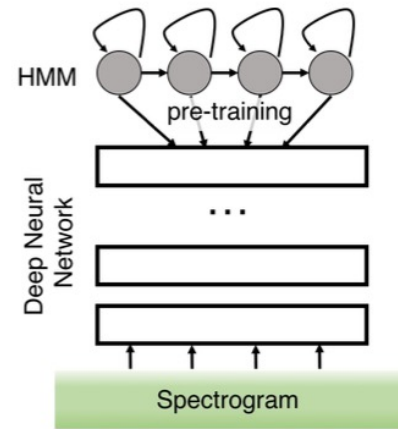
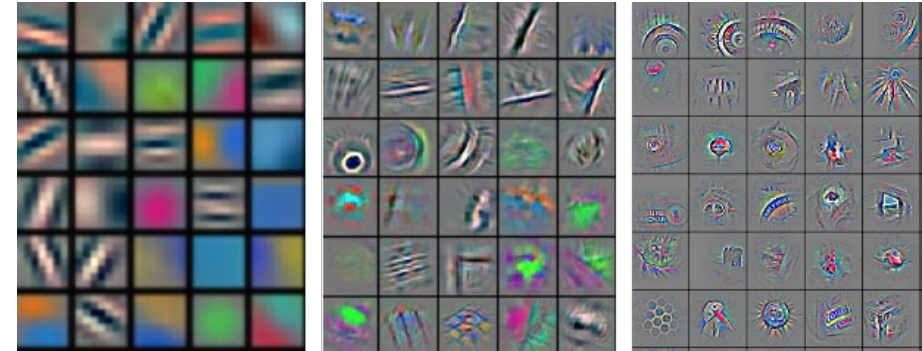


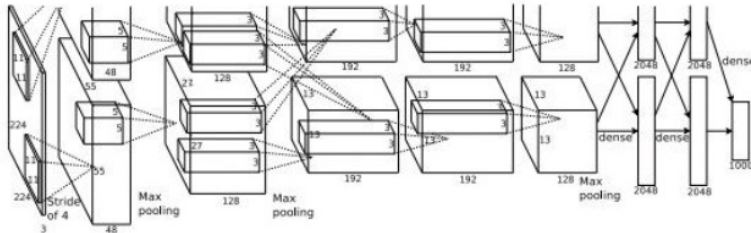
Illustration of Dahl et al. 2012 by Lane McIntosh, copyright CS231n 2017



Visual features extracted in different layers in CNN

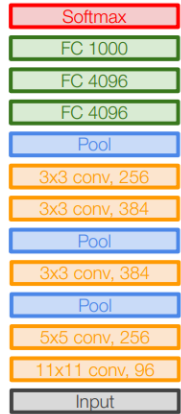
Imagenet classification with deep convolutional neural networks

Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton, 2012



Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

From Neural Networks to “Deep Learning”

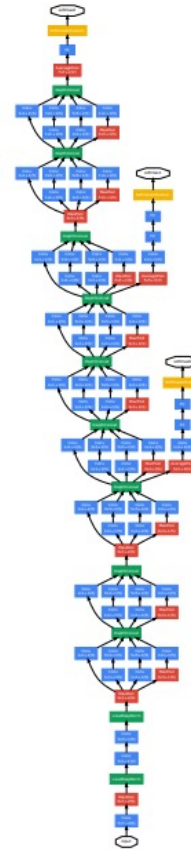


AlexNet

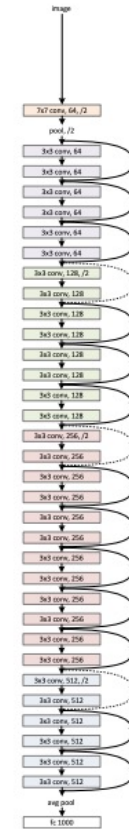


VGG16

VGG19



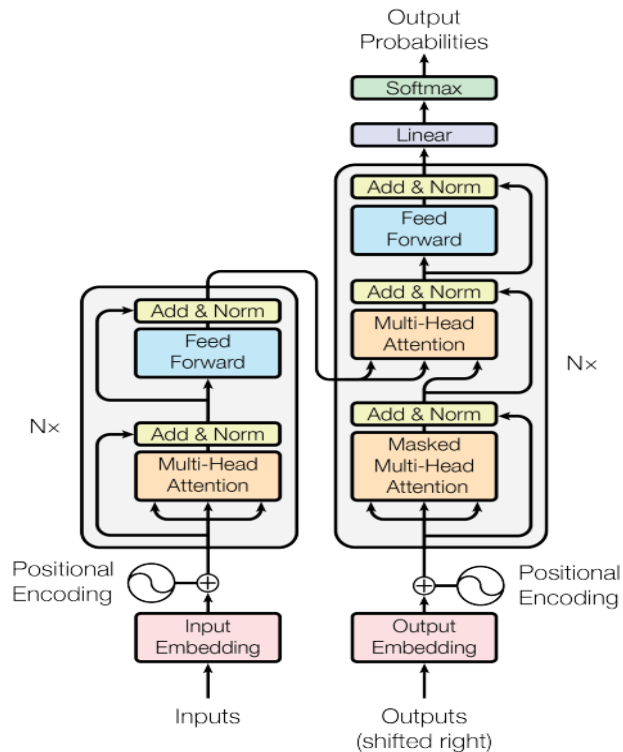
GoogleNet (Szegedy et al., 2015)



ResNet (He et al., 2015)

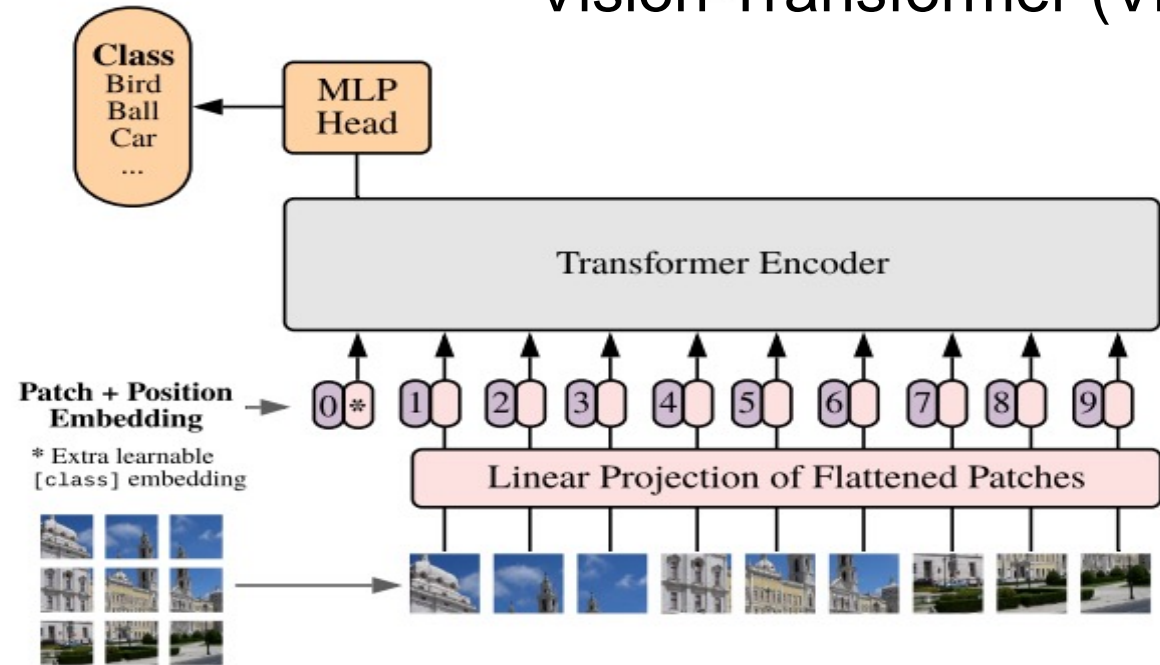
From Neural Networks to “Deep Learning”

Transformer



(Vaswani et al., 2017)

Vision Transformer (ViT)



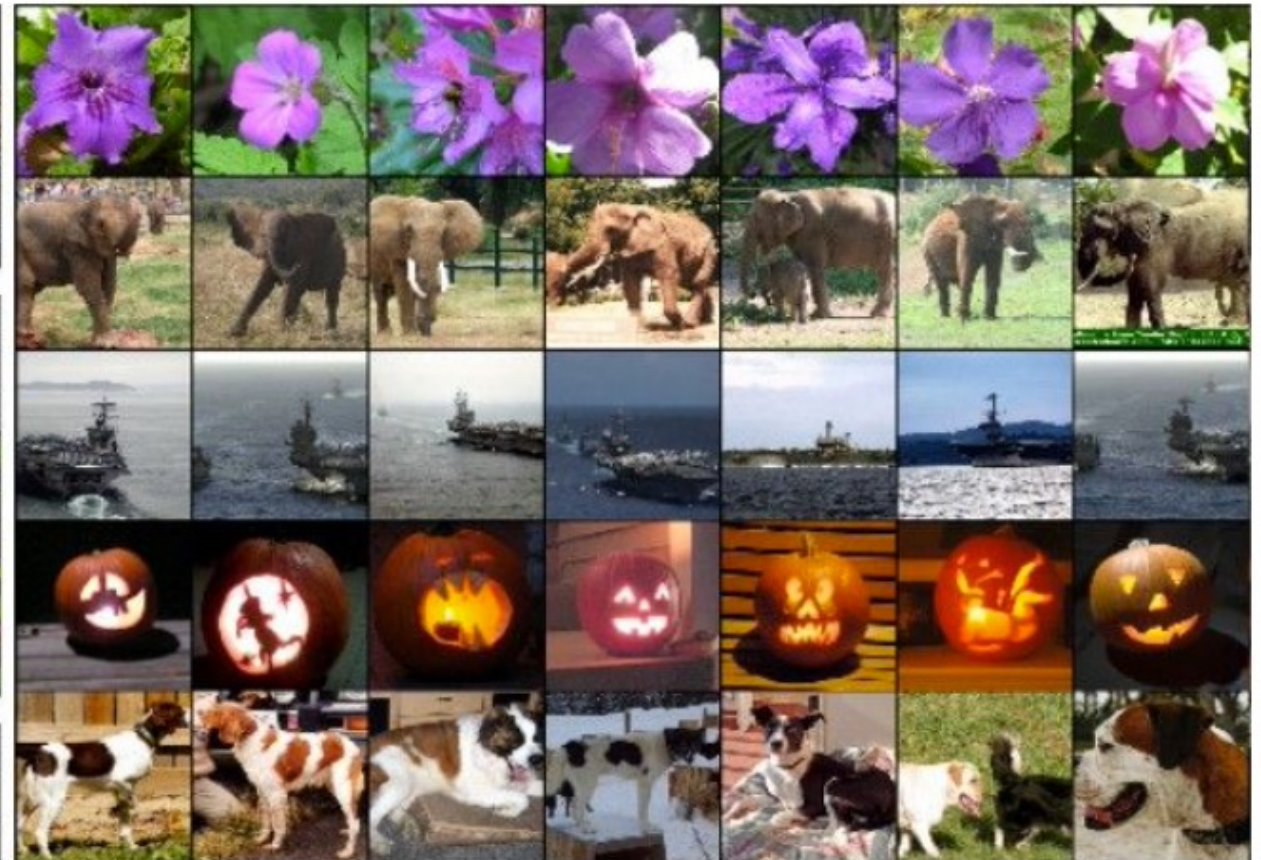
Dosovitskiy, Alexey, et al. "An image is worth 16x16 words: Transformers for image recognition at scale." *arXiv preprint arXiv:2010.11929* (2020).

From Neural Networks to “Deep Learning”

Classification



Retrieval



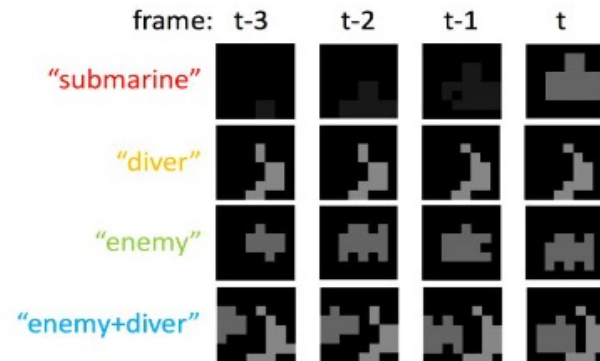
Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

From Neural Networks to “Deep Learning”



Images are examples of pose estimation, not actually from Toshev & Szegedy 2014. Copyright Lane McIntosh.

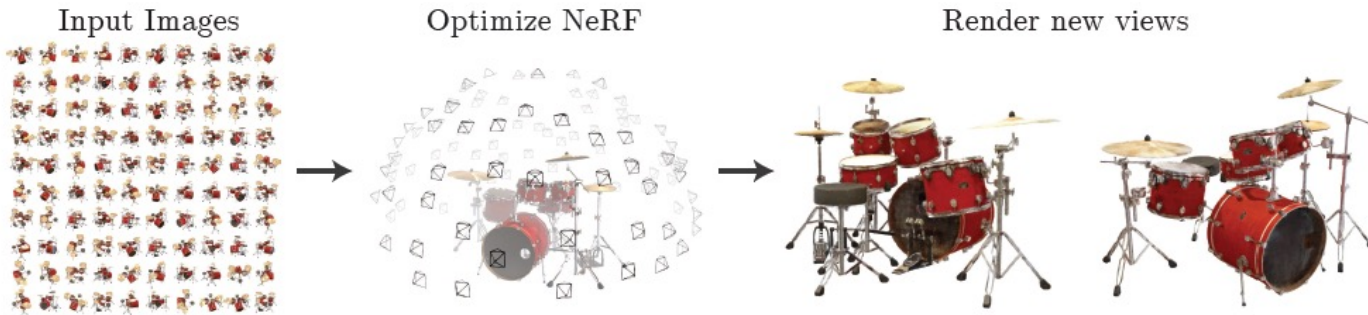
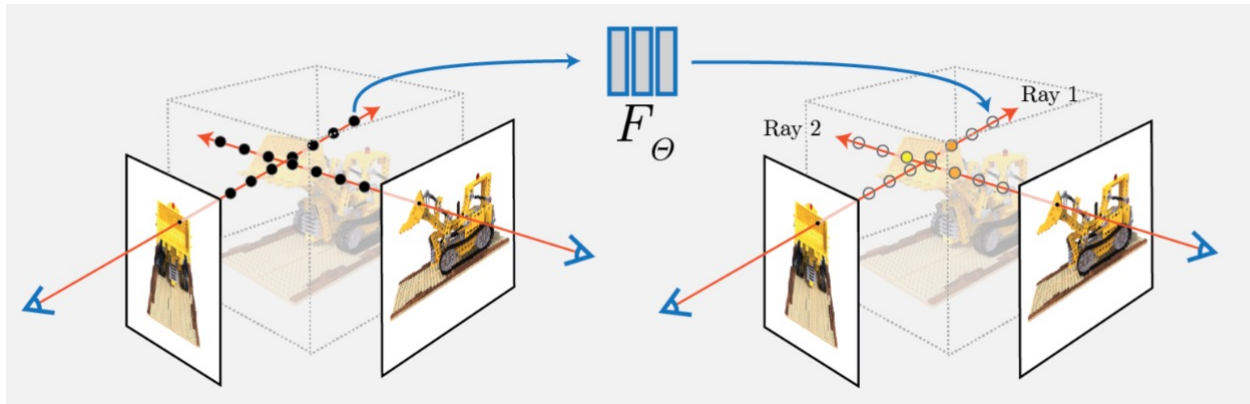
[Toshev, Szegedy 2014]



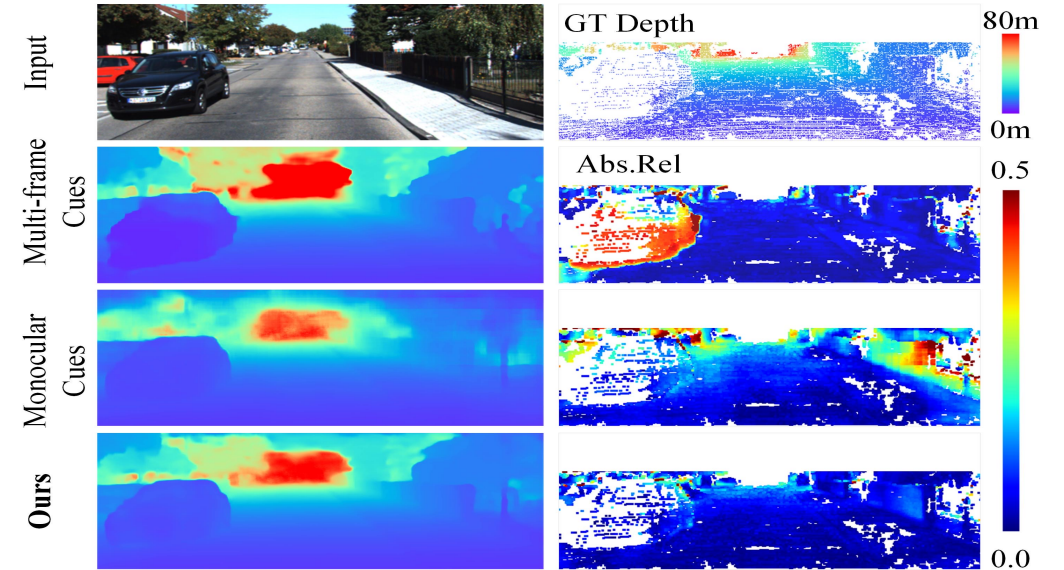
[Guo et al. 2014]

Figures copyright Xiaoxiao Guo, Satinder Singh, Honglak Lee, Richard Lewis, and Xiaoshi Wang, 2014. Reproduced with permission.

From Neural Networks to “Deep Learning”



Neural Radiance Fields (NeRF) for 3D vision
<https://www.matthewtancik.com/nerf>



Deep learning for depth estimation
<https://ruili3.github.io/dymultidepth/index.html>

From Neural Networks to “Deep Learning”



self-driving cars

Photo by Lane McIntosh. Copyright CS231n 2017.

This image by Christin Khan is in the public domain and originally came from the U.S. NOAA.



Whale recognition, Kaggle Challenge

Photo and figure by Lane McIntosh; not actual example from Mnih and Hinton, 2010 paper.



Mnih and Hinton, 2010



Original image is CC0 public domain
Starry Night and Tree Roots by Van Gogh are in the public domain
Bokeh image is in the public domain
Stylized images copyright Justin Johnson, 2017;



Gatys et al, "Image Style Transfer using Convolutional Neural Networks", CVPR 2016
Gatys et al, "Controlling Perceptual Factors in Neural Style Transfer", CVPR 2017



<https://github.com/donggong1/learn-optimizer-rgdn>
<https://donggong1.github.io/blur2mflow.html>

From Neural Networks to “Deep Learning”



A white teddy bear sitting in the grass



A man in a baseball uniform throwing a ball



A woman is holding a cat in her hand



A man riding a wave on top of a surfboard

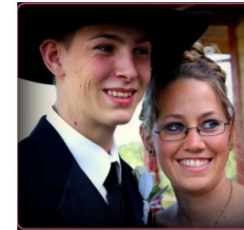


A cat sitting on a suitcase on the floor

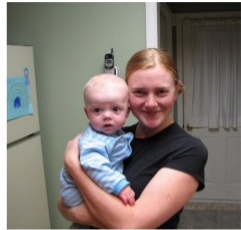


A woman standing on a beach holding a surfboard

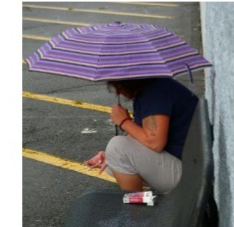
Who is wearing glasses?
man woman



Where is the child sitting?
fridge arms



Is the umbrella upside down?
yes no



How many children are in the bed?
2 1



Image Captioning. Vinyals et al, 2015 Karpathy and Fei-Fei, 2015

Vision question answering (VQA)

From Neural Networks to “Deep Learning”

TEXT PROMPT

an armchair in the shape of an avocado. an armchair imitating an avocado.

AI-GENERATED IMAGES



Ramesh et al, “DALL·E: Creating Images from Text”, 2021. <https://openai.com/blog/dall-e/>

“A raccoon astronaut with the cosmos reflecting on the glass of his helmet dreaming of the stars”



Generated by DALL·E 2

Convolutional Neural Network (CNN)

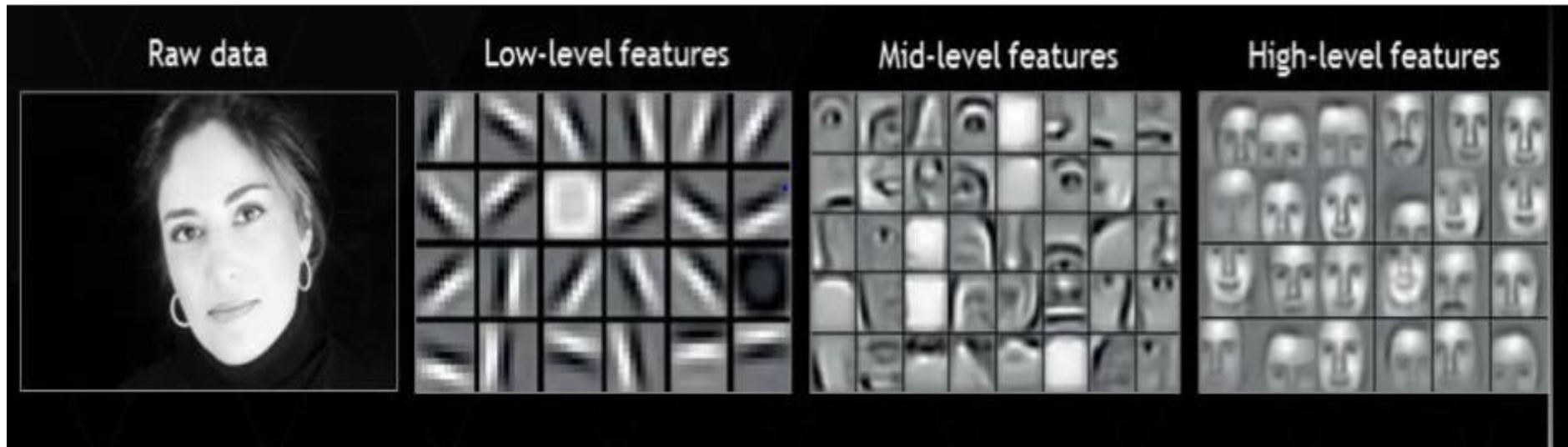
Convolutional Neural Networks (CNNs)

- Convolutional neural networks (CNNs) is a **class of deep learning methods** which has become dominant in various computer vision tasks and are **suitable for processing 2D or 3D data such as images and videos**.
- Convolutional neural network is composed of **multiple building blocks**, such as **convolution layers**, **pooling layers**, and **fully connected layers**, and is designed to automatically and adaptively learn spatial hierarchies of features through a backpropagation algorithm.

<http://www.analyticsvidhya.com/blog/2017/04/comparison-between-deep-learning-machine-learning/>

CNNs

- CNNs can be interpreted as **gradually transforming the images into a representation** in which the classes are separable by a linear classifier.
- CNNs will try to **learn low-level features such as edges and lines in early layers**, then parts of objects and then **high-level representation of an object in subsequent layers**.

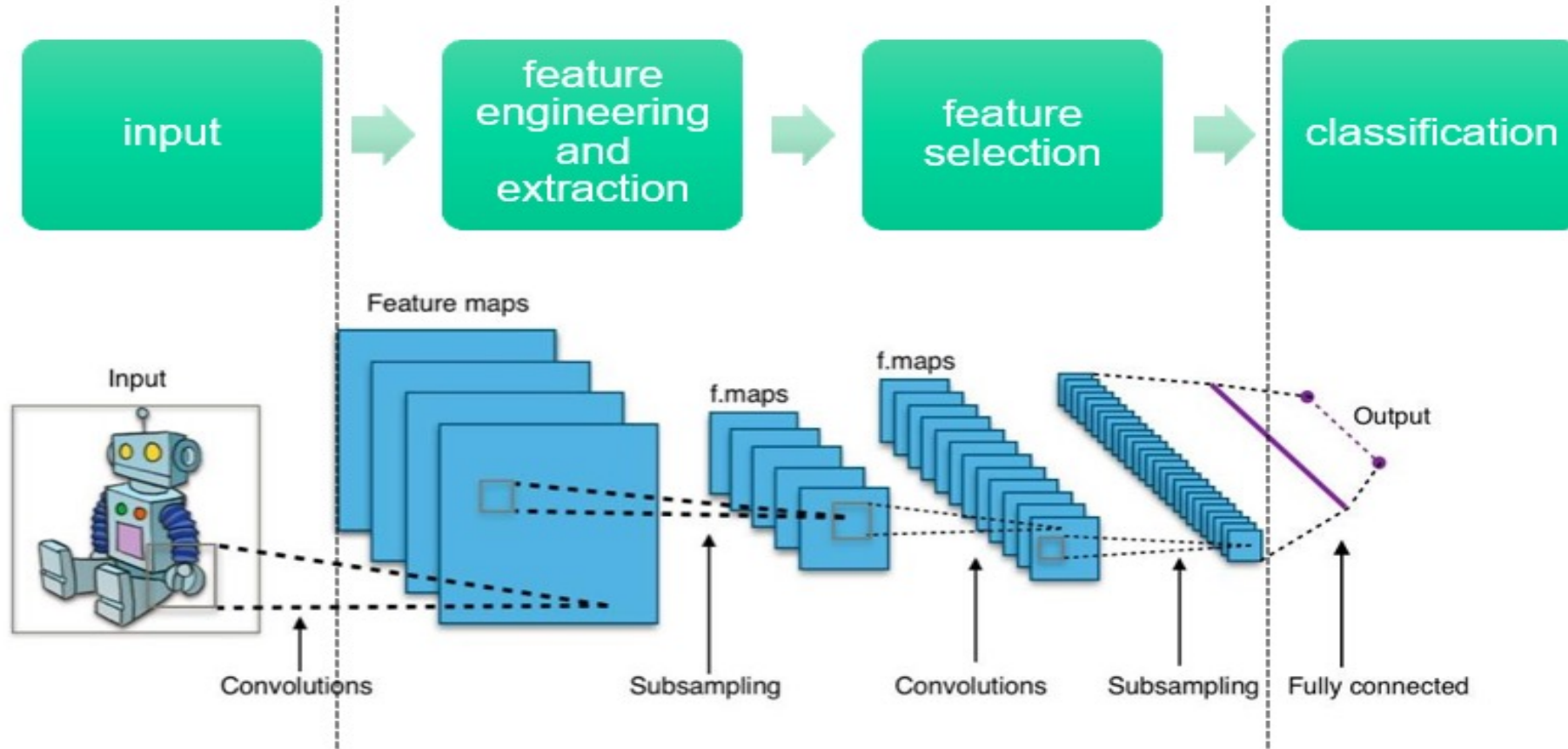


<http://www.analyticsvidhya.com/blog/2017/04/comparison-between-deep-learning-machine-learning/>

CNNs

- CNNs are used for image recognition, object detection, and classification tasks.
- CNNs excel at analysing complex visual data, such as images and videos. **CNNs can automatically detect and recognize patterns, shapes, and objects within images.**
- They have **spatial invariance**, allowing them to recognize objects regardless of location or orientation in an image.
- CNNs excel at feature extraction, learning complex and abstract features from input data.
- **Parameter sharing in CNNs reduces computational and memory requirements**, which makes them efficient.
- They can be pre-trained on large datasets and fine-tuned for specific tasks, enabling transfer learning.
- CNNs are scalable and can handle inputs of different sizes.

Traditional Approach vs DL Approach



<https://towardsdatascience.com/convolutional-neural-networks-for-all-part-i-cdd282ee7947>

From Neural Networks to “Deep Learning”

Core ideas go back many decades!

The **Mark I Perceptron** machine was the first implementation of the perceptron algorithm.

The machine was connected to a camera that used 20×20 cadmium sulfide photocells to produce a 400-pixel image.

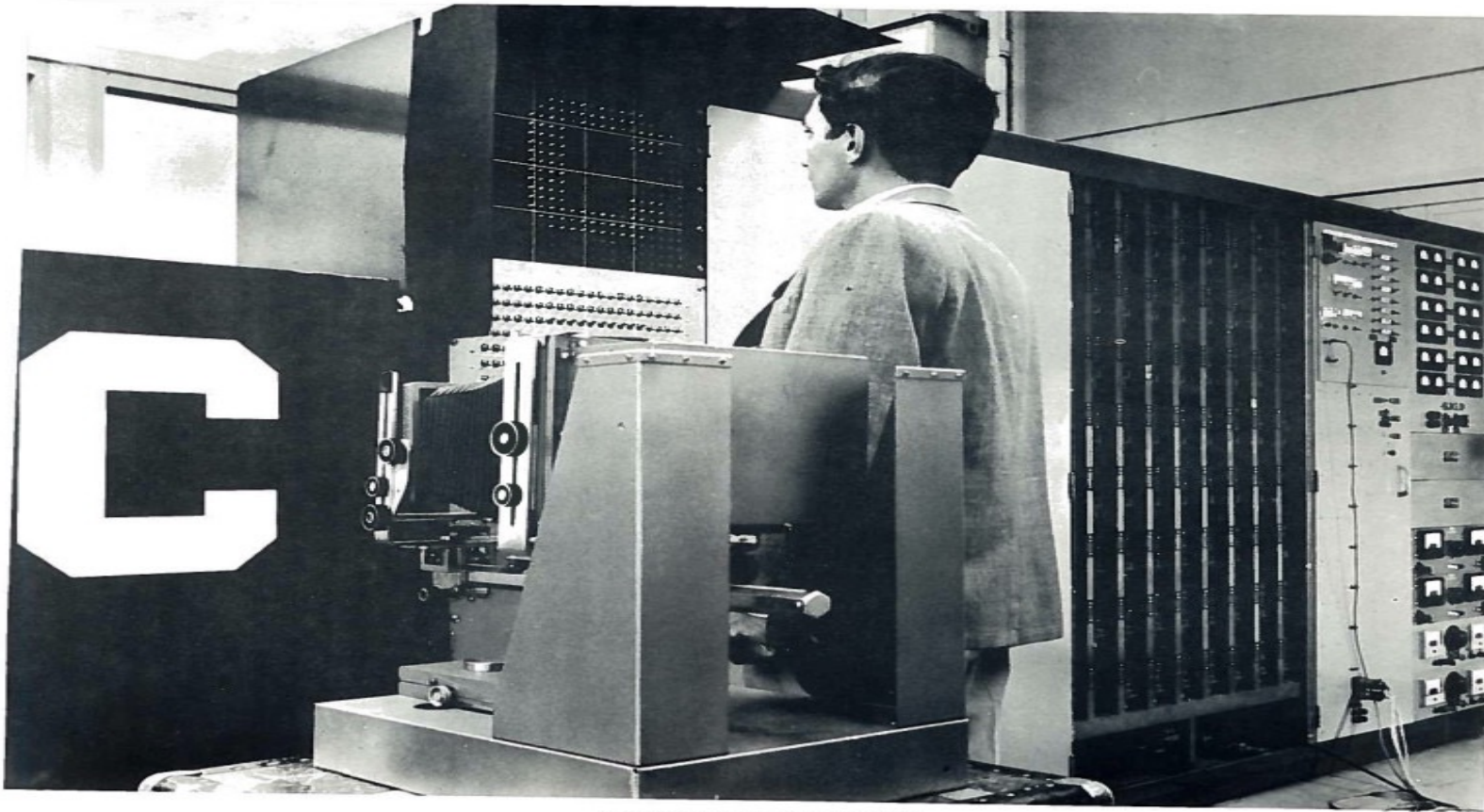
recognized
letters of the alphabet

Frank Rosenblatt, ~1957: Perceptron



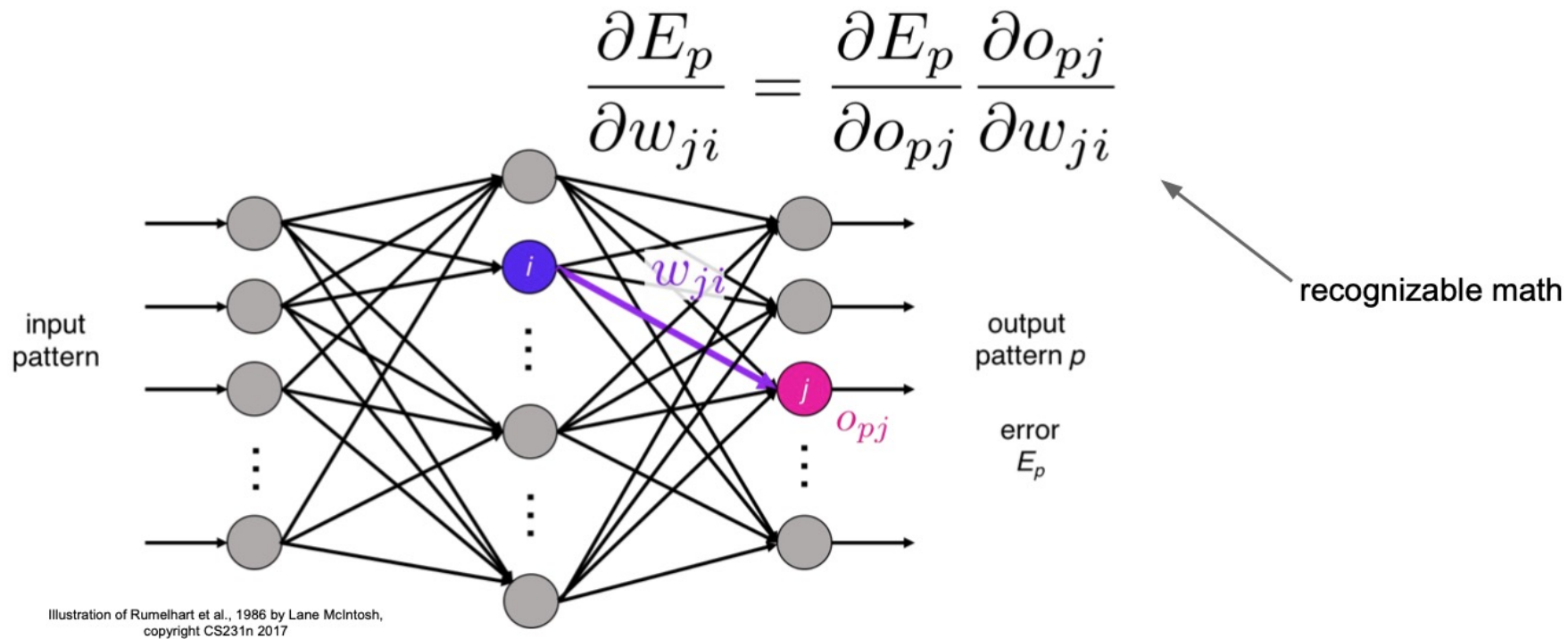
[This image](#) by Rocky Acosta is licensed under [CC-BY 3.0](#)

From Neural Networks to “Deep Learning”



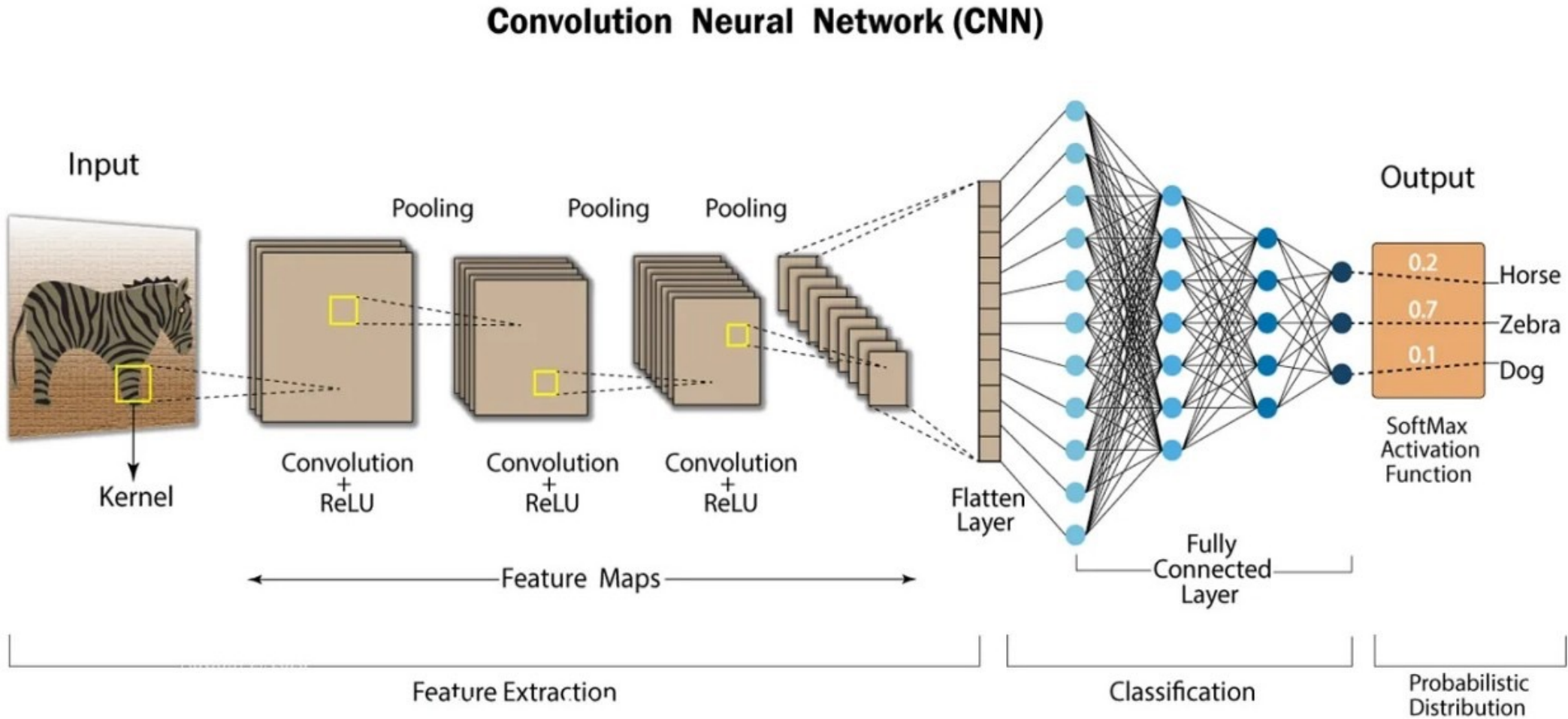
THE MARK I PERCEPTRON

From Neural Networks to “Deep Learning”

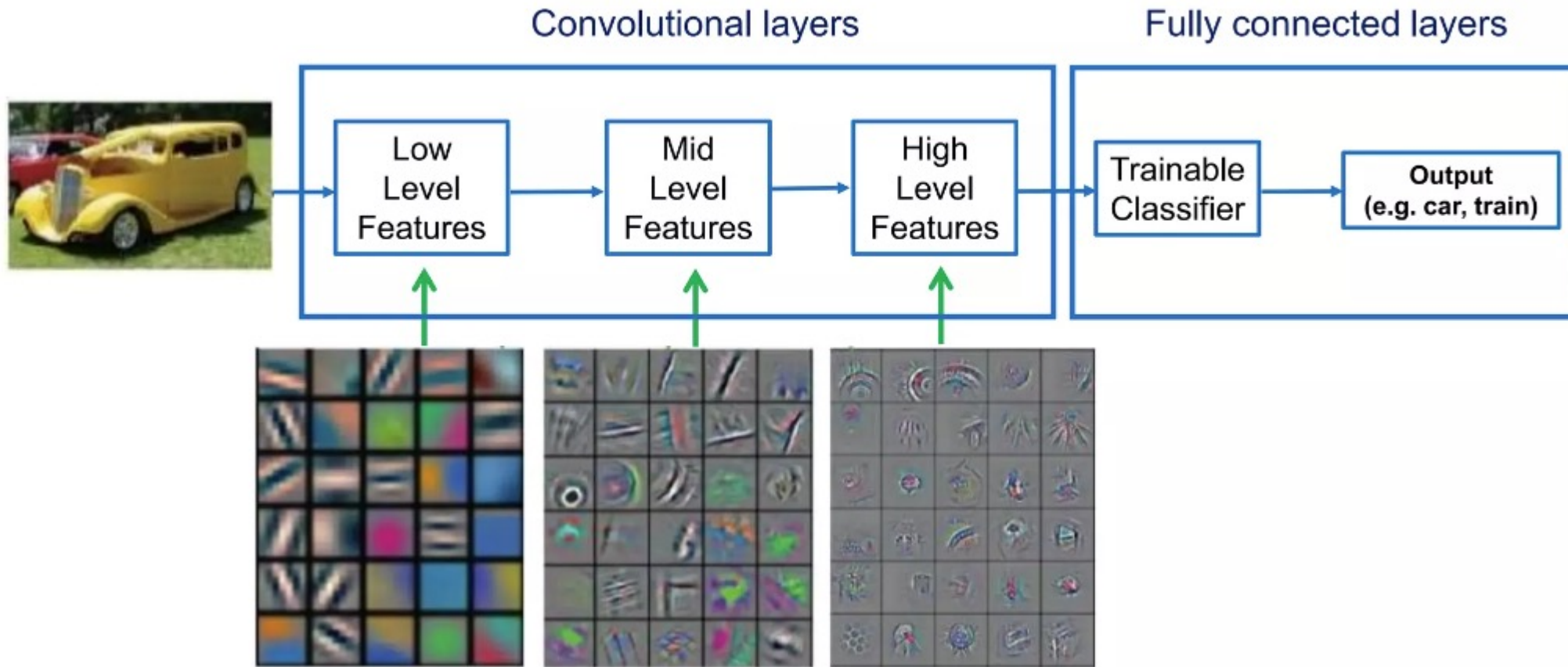


Rumelhart et al., 1986: First time back-propagation became popular

Convolutional Neural Network (CNN)



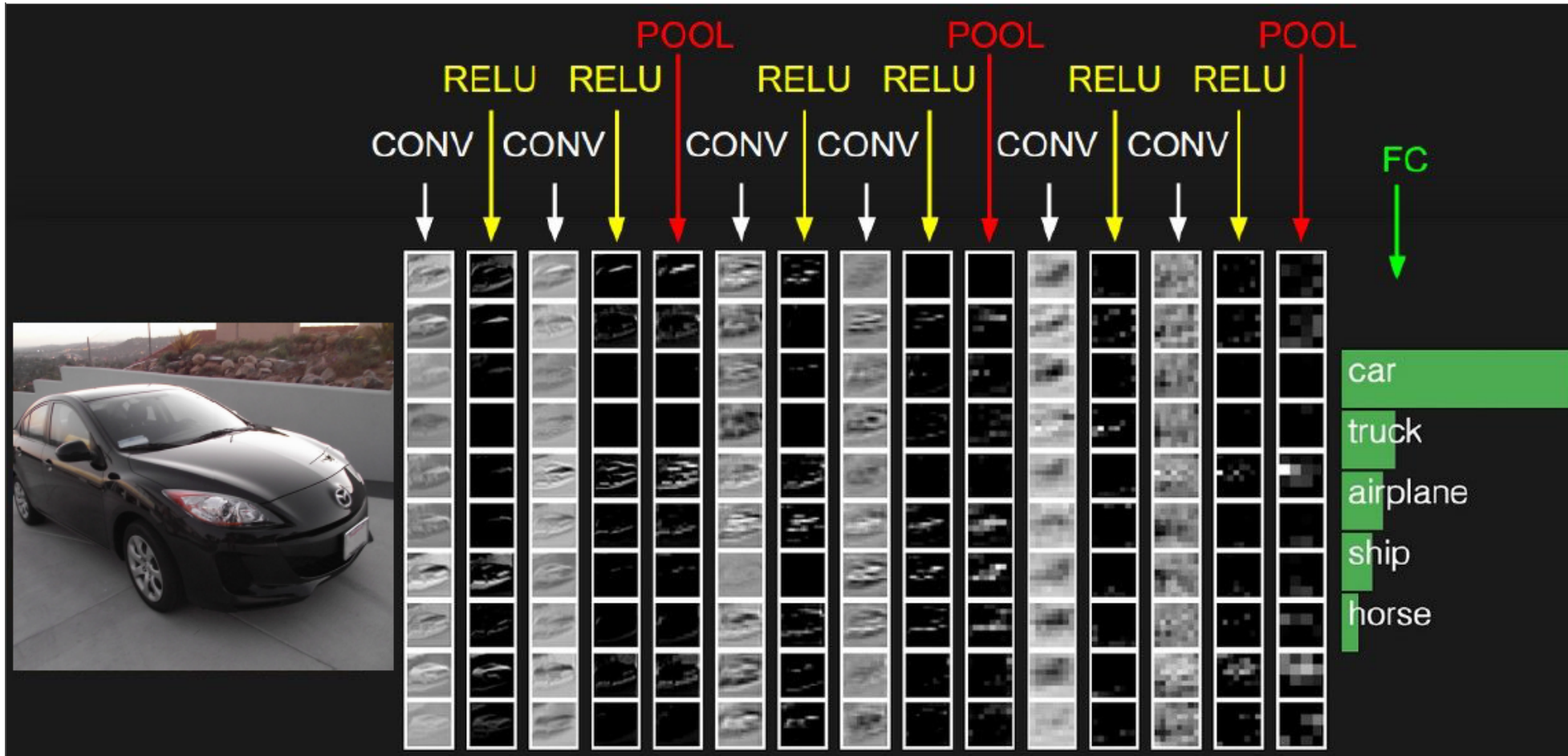
CNNs- What do they learn?



<https://www.slideshare.net/NirthikaRajendran/cnn-126271677>

[From recent Yann LeCun slides]

CNN-Components

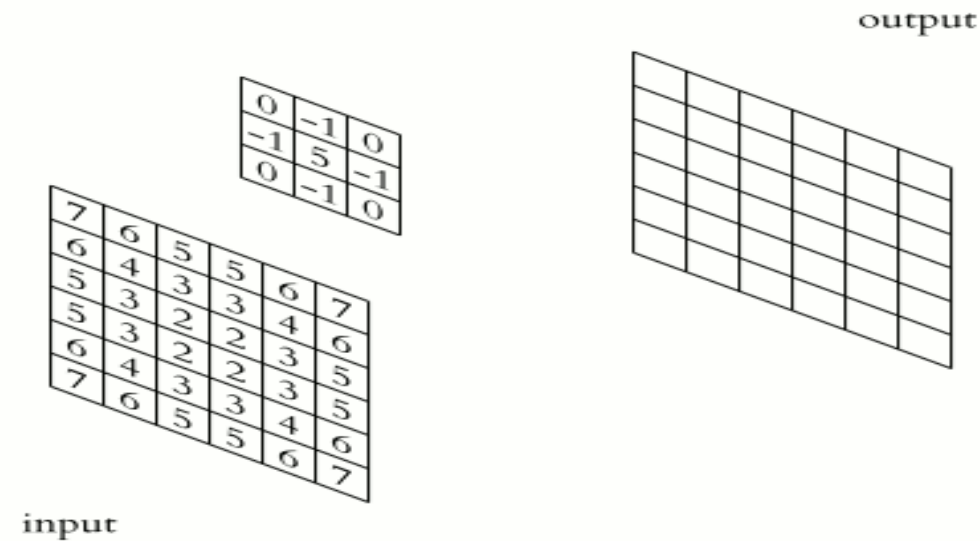


CNN building blocks

- Convolution layer
- ReLU activation
- Subsampling or Pooling layer
- Flattening
- Fully connected layer
- Output Layer

Convolution layer

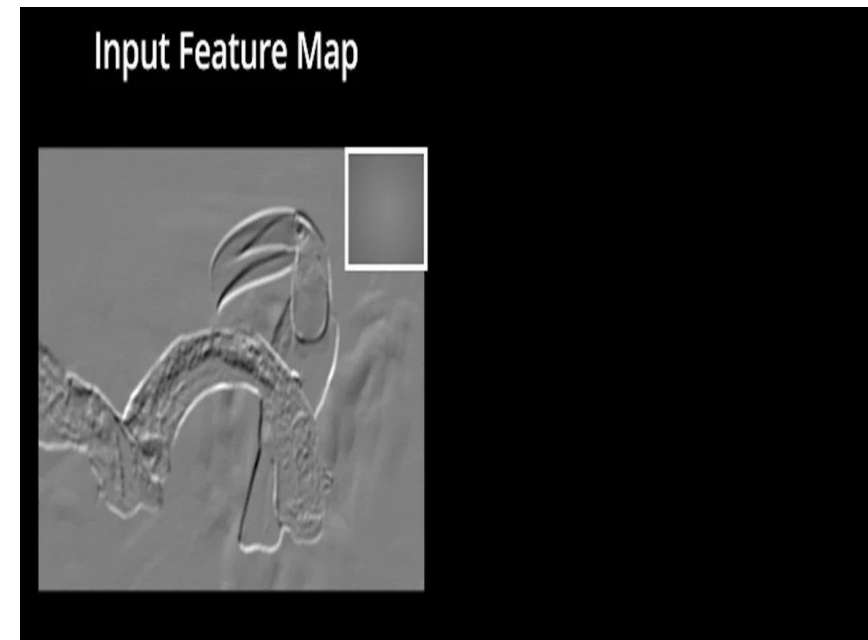
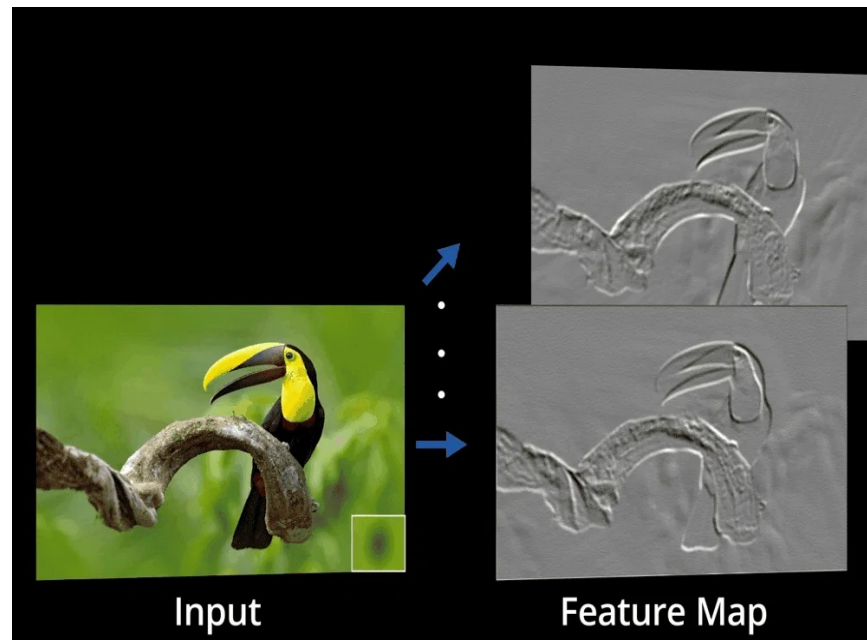
- Convolution layer extracts valuable features from an image. A convolution layer has several filters that perform the convolution operation.



ReLU activation

ReLU stands for the **rectified linear unit**. Once the feature maps are extracted, the next step is to move them to a ReLU layer.

ReLU performs an element-wise operation and sets all the negative pixels to 0. It introduces non-linearity to the network, and the generated output is a rectified feature map.

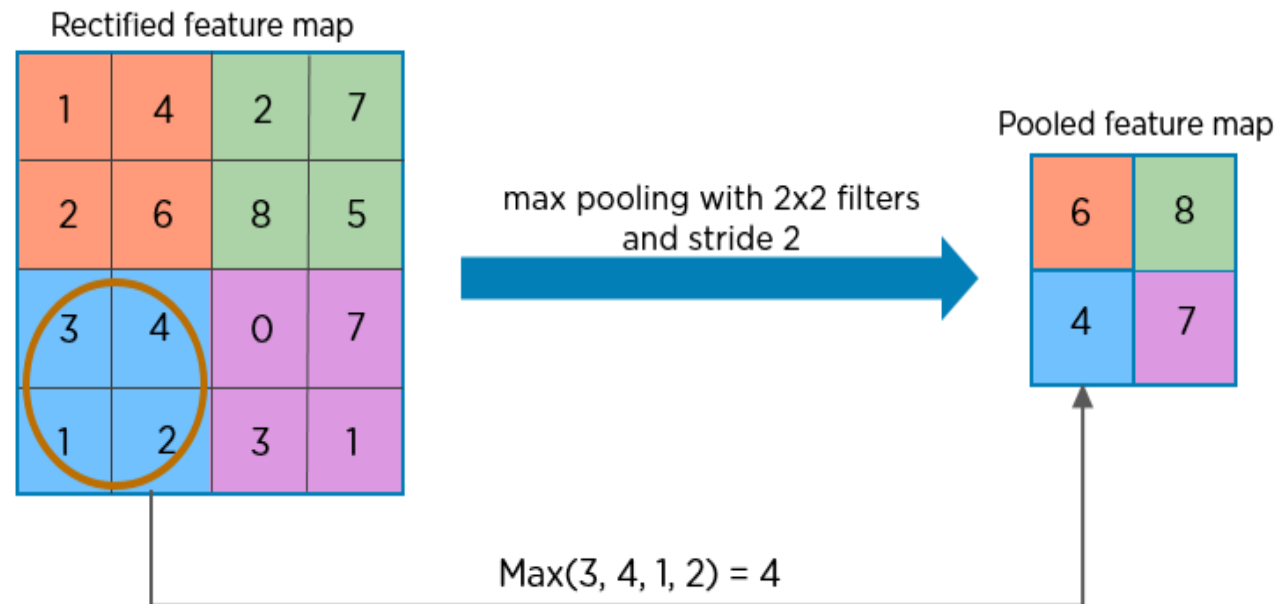


Pooling layer

Pooling is a down-sampling operation that reduces the dimensionality of the feature map.

The rectified feature map now goes through a pooling layer to generate a pooled feature map.

The pooling layer uses various filters to identify different parts of the image like edges, corners, body, feathers, eyes, and beak.

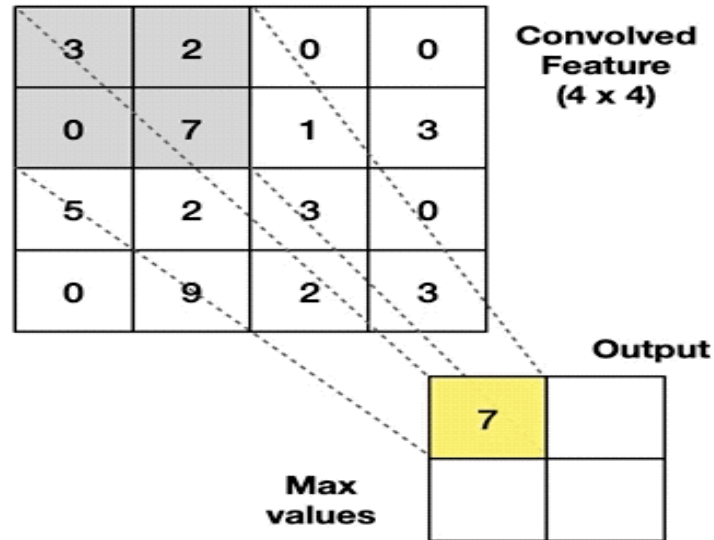


Pooling layer

Max Pooling

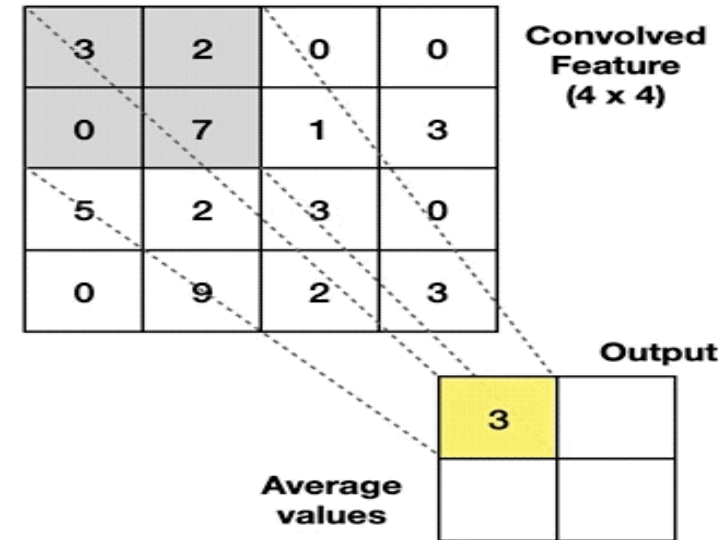
Take the **highest** value from the area covered by the kernel

Example: Kernel of size 2 x 2; stride=(2,2)



Average Pooling

Calculate the **average** value from the area covered by the kernel

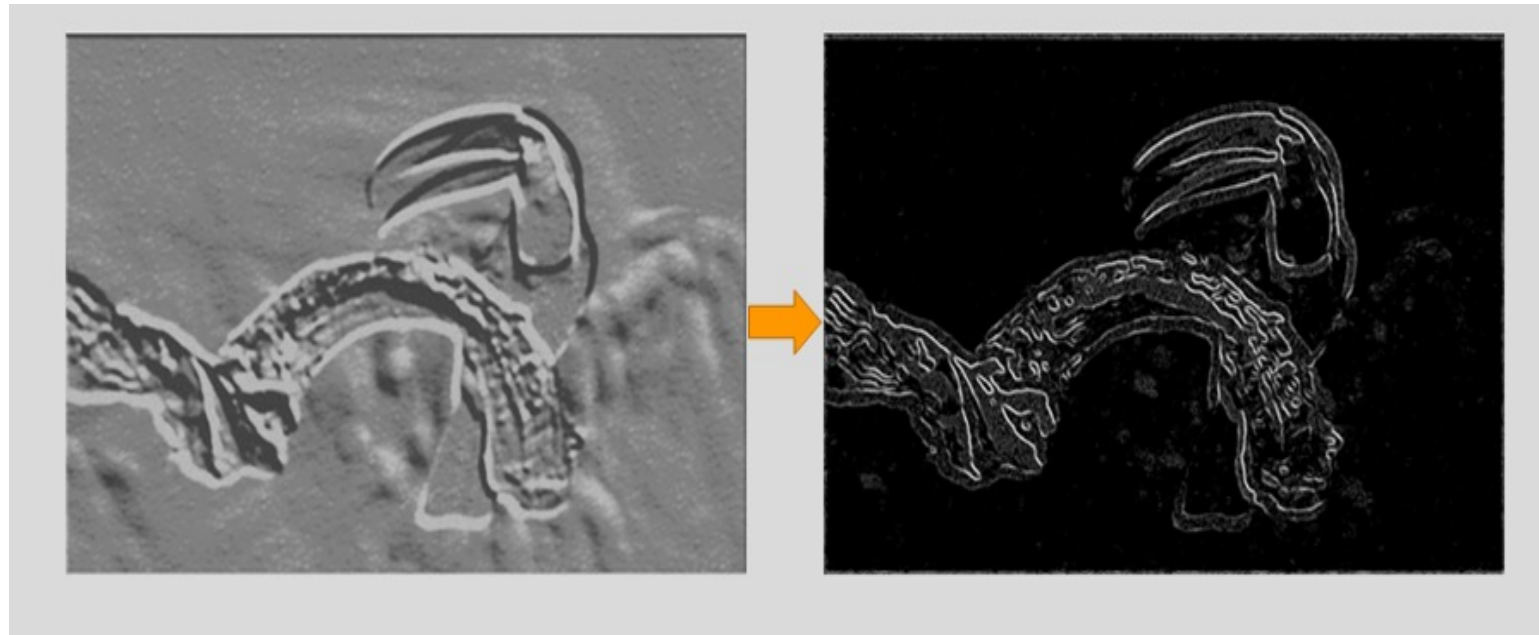


Medium: Intro to Pooling layers in CNN <https://pub.towardsai.net/introduction-to-pooling-layers-in-cnn-dafe61eabe34>

Pooling layer

Pooling is a down-sampling operation that reduces the dimensionality of the feature map. The rectified feature map now goes through a pooling layer to generate a pooled feature map.

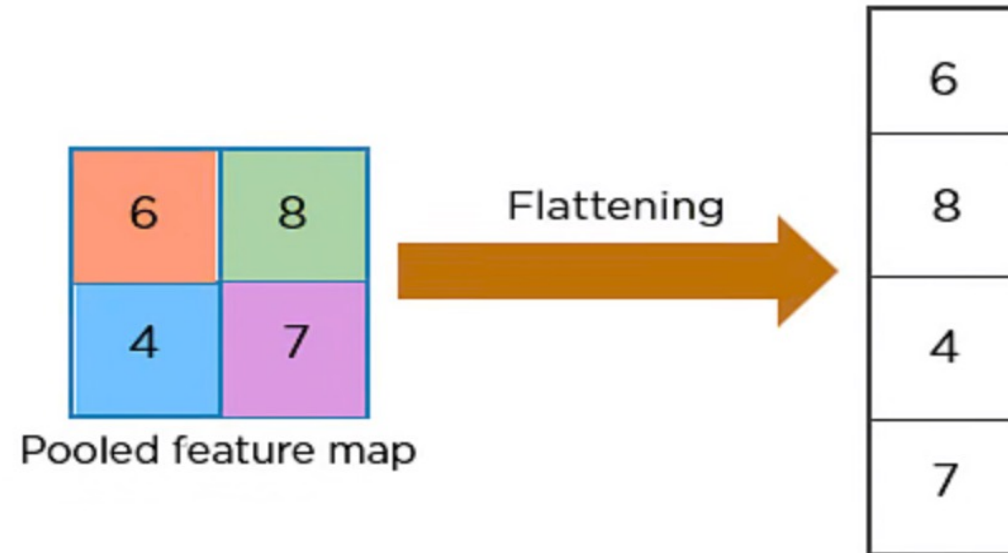
The pooling layer uses various filters to identify different parts of the image like edges, corners, body, feathers, eyes, and beak.



Flattening

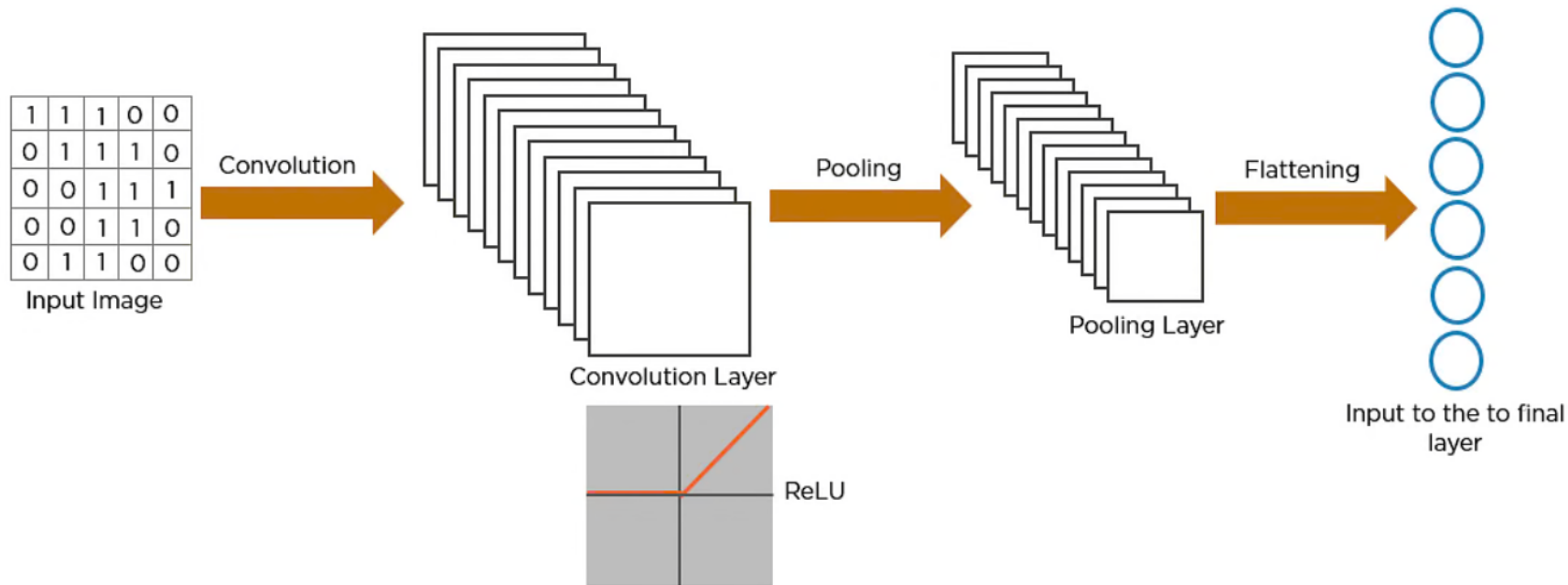
After the convolution and pooling operations, the feature maps still exist in a multi-dimensional format. **Flattening converts these feature maps into a one-dimensional vector.**

This process is essential because it **prepares the data to be passed into fully connected layers for classification or regression tasks.**



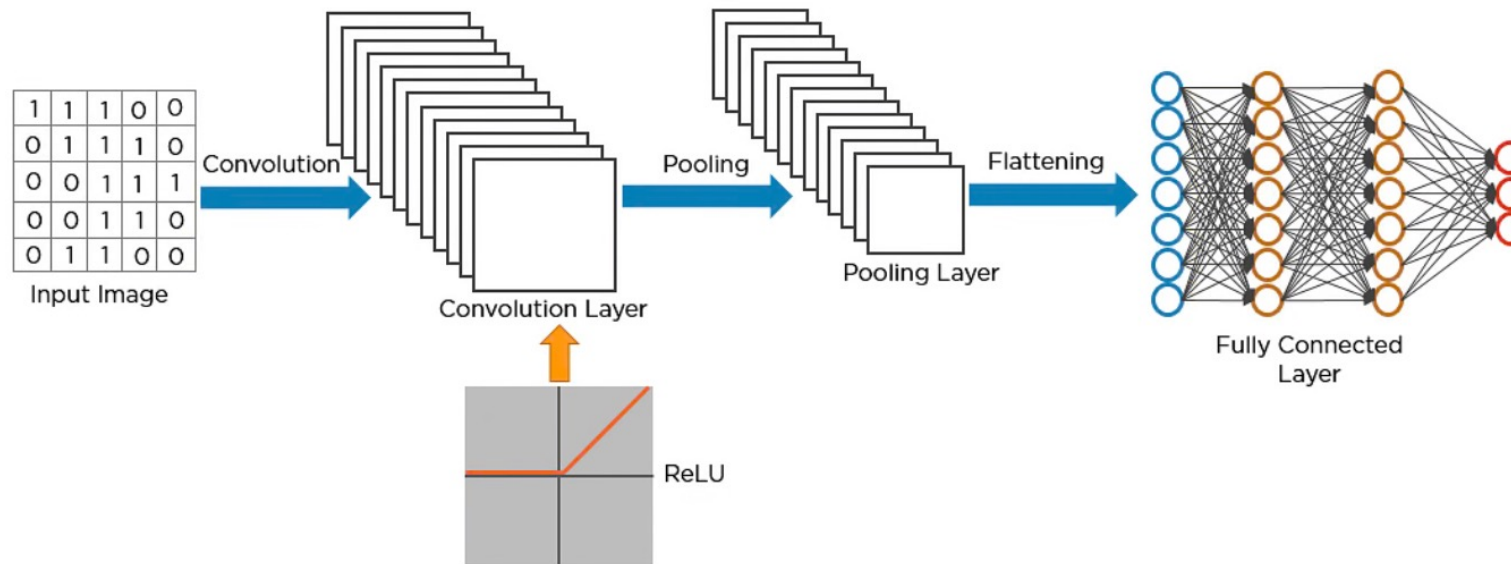
Conv + ReLU + Pooling ...

You can have multiple blocks of (Conv + ReLU + Pooling) before we reach fully connected layers



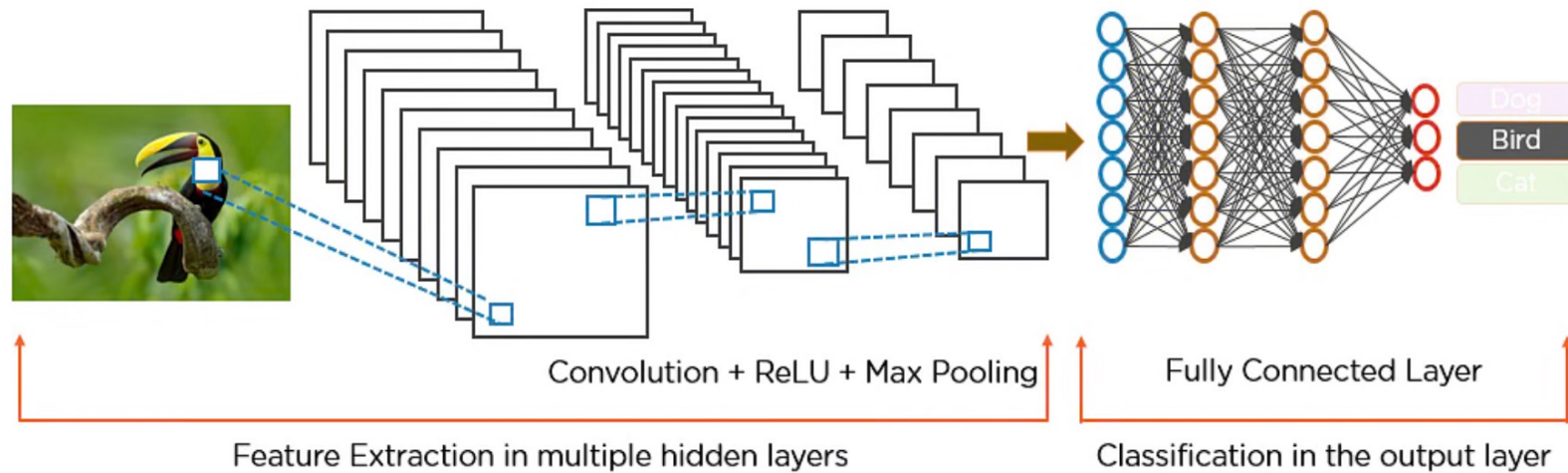
Fully Connected (FC) Layers

- The FC layer comprises the weights and biases together with the neurons and is used to connect the neurons between two separate layers.
- **In FC layer, each node in the output layer connects directly to a node in the previous layer.**
- The flattened feature map is passed through the FC layer(s)



Output Layer

- The output layer **produces the probability of each class** given the input image
- This is **the last layer containing the same number of neurons as the number of classes in the dataset**
- The output of this layer passes through the **Softmax activation function** to normalize the output to have probability sum to one

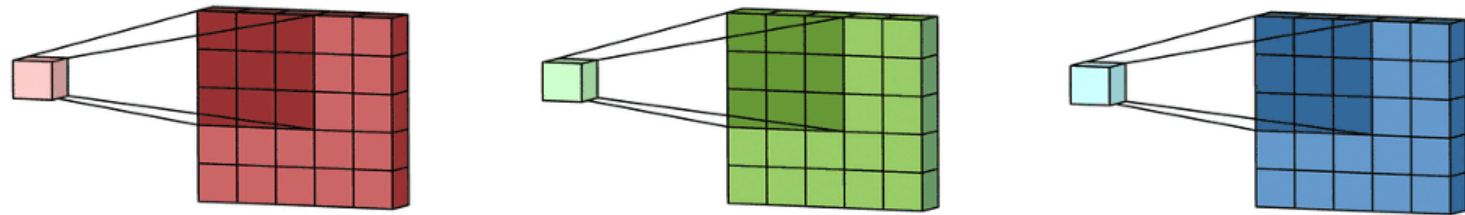


Convolution operator parameters

- Channels
- Filter size
- Padding
- Stride
- Dilation
- Activation function

Channels

- For an RGB image, there are typically **separate kernels for each colour channel** because different features might be more visible or relevant in one channel compared to the others.
- Each filter produces a separate **feature map**, and the collection of these feature maps forms the ***complete output of the layer***.
- The output normally has multiple channels, where each channel is a feature map corresponding to a particular kernel.



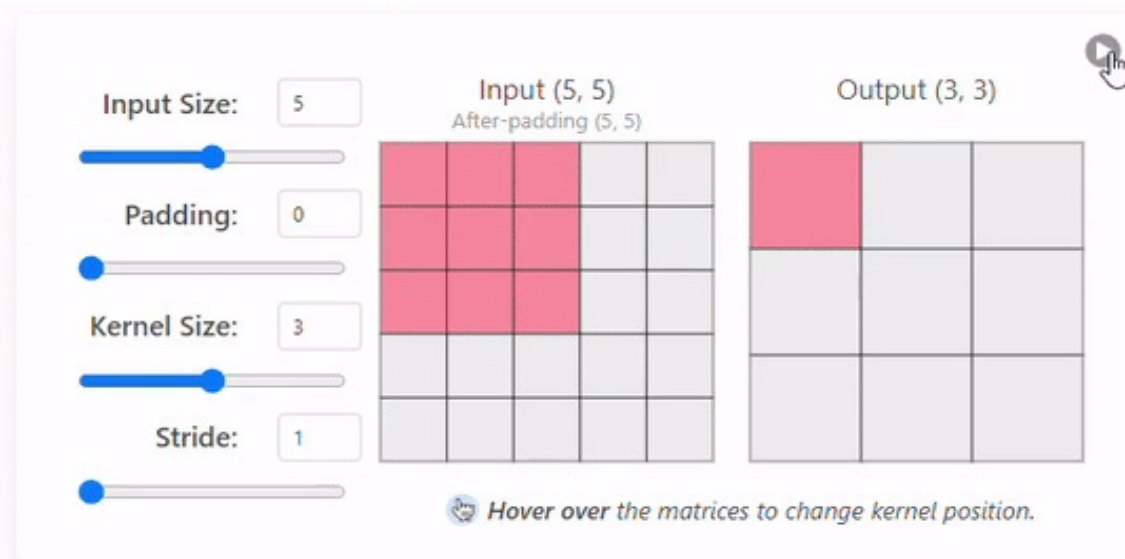
Filter size

- Filter size can be 5 by 5, 3 by 3, and so on
- Larger filter sizes should be avoided in many cases (**not always!**)
 - As learning algorithm needs to learn filter values (weights)
- Odd sized filters are used more often than even sized filters (**not always!**)
 - Nice geometric property of all input pixels being around output pixel

Stride

- How many pixels we move filter to the right/down is stride
- Stride 1: move filter one pixel to the right/down
- Stride 2: move filter two pixels to the right/down

Understanding Hyperparameters

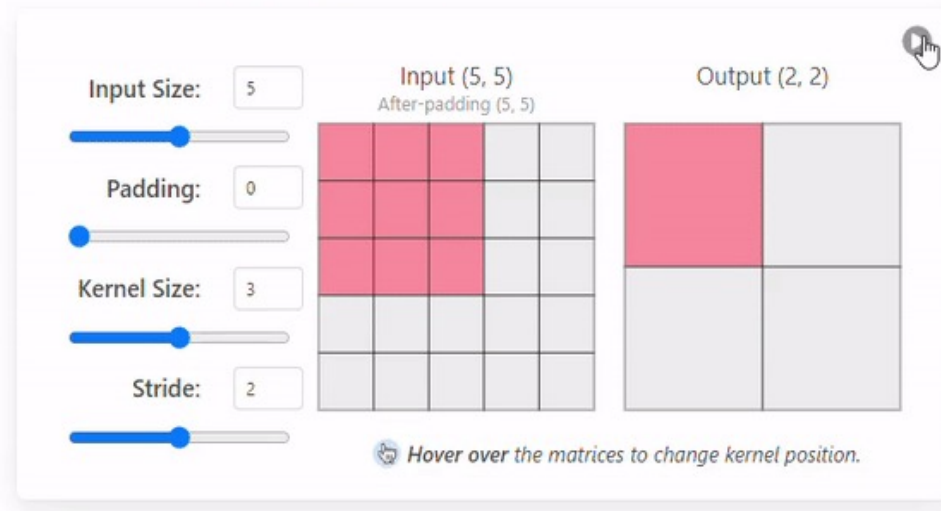


<https://poloclub.github.io/cnn-explainer/>

Stride

- A **larger stride** will produce smaller output dimensions (as it covers the input image faster), whereas a **smaller stride** results in a larger output dimension.
- **Increasing** the stride will allow the filter to cover a **larger area of the input image**, which can be useful for capturing **more global features**. In contrast, **lowering** the stride will capture **finer and more local details**.

Understanding Hyperparameters

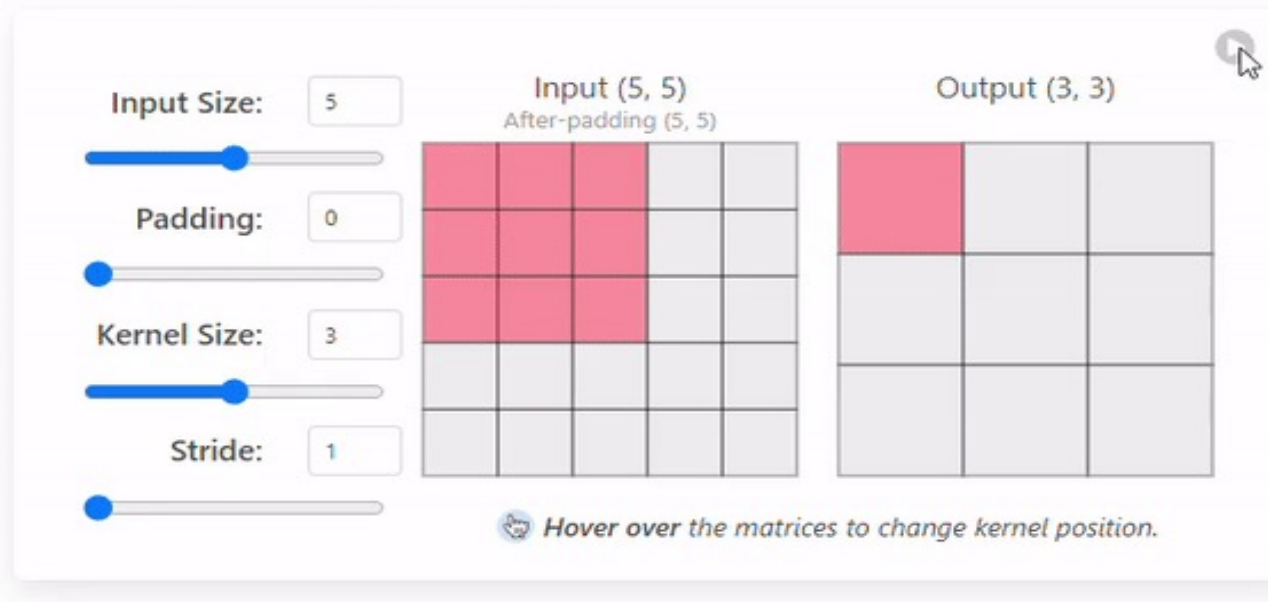


<https://poloclub.github.io/cnn-explainer/>

Padding

- Padding refers to the **addition of extra pixels around the edge** of the input image.
- The purpose of padding is to **adjust the spatial size** of the output of a convolutional operation and to **preserve spatial information at the borders**.
- Padding = 0

Understanding Hyperparameters

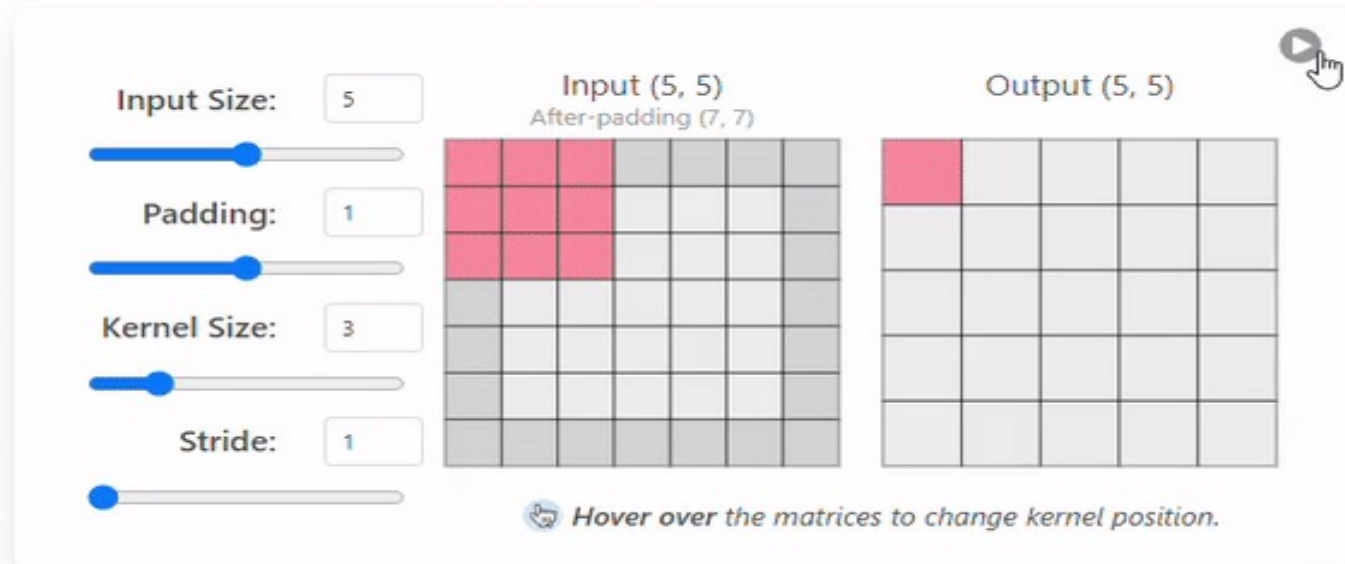


<https://poloclub.github.io/cnn-explainer/>

Padding

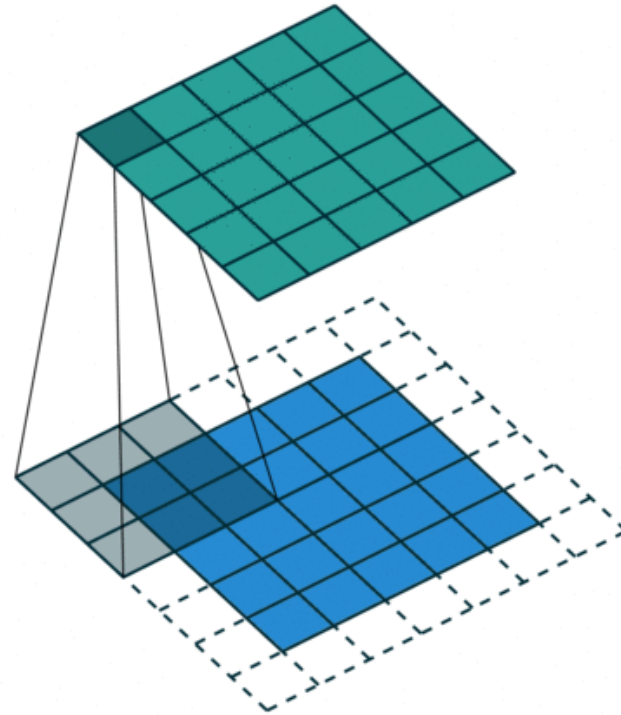
- When to apply padding: Mainly when the edges of the image **contain useful information** that you want to capture.
- Padding **increases the size of the output feature map**.
- Padding = 1

Understanding Hyperparameters



<https://poloclub.github.io/cnn-explainer/>

3 by 3 filter with padding of 1

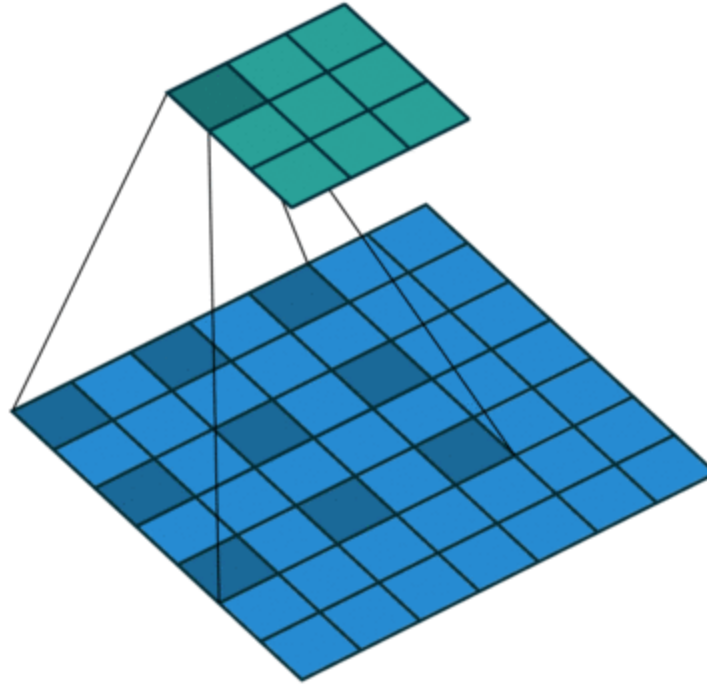


<https://training.galaxyproject.org/training-material/topics/statistics/tutorials/CNN/slides-plain.html>

Dilation

- When we apply 3 by 3 filter, output affected by pixels in 3 by 3 subset of image
- Dilation: To have a larger receptive field (portion of image affecting filter's output)
- If dilation set to 2, instead of contiguous 3 by 3 subset of image, every other pixel of a 5 by 5 subset of image affects output

3 by 3 filter with dilation of 2



<https://training.galaxyproject.org/training-material/topics/statistics/tutorials/CNN/slides-plain.html>

Activation function

- After filter applied to whole image, apply activation function to output to introduce non-linearity
- Preferred activation function in CNN is ReLU
- ReLU leaves outputs with positive values as is, replaces negative values with 0

0	1	-4
0	2	0
-1	4	3

Filter output

-->

0	1	0
0	2	0
0	4	3

Filter output after ReLU

Single channel 2D convolution

0	2	1	0	1
0	1	2	1	0
1	0	2	0	0
1	0	0	1	1
0	1	1	2	2

Input Vector

0	1	-4
0	2	0
-1	4	3

Filter

5		

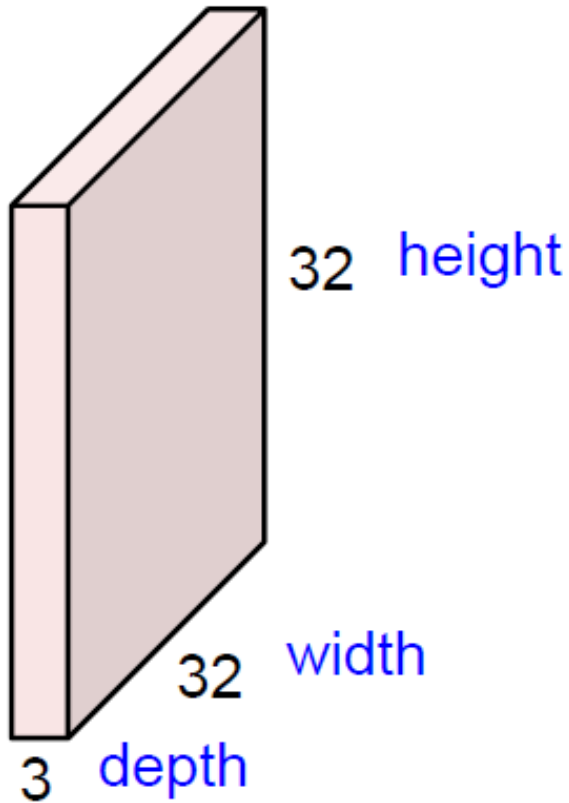
Output

$$\begin{aligned} &= 0 \times 0 + 2 \times 1 + 1 \times -4 + \\ &\quad 0 \times 0 + 1 \times 2 + 2 \times 0 + \\ &\quad 1 \times -1 + 0 \times 4 + 2 \times 3 = 5 \end{aligned}$$

<https://training.galaxyproject.org/training-material/topics/statistics/tutorials/CNN/slides-plain.html>

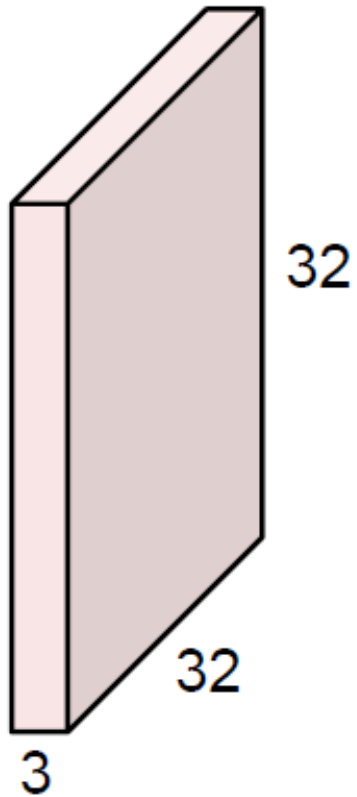
CNN: Convolutional Layer

32x32x3 image -> preserve spatial structure



CNN: Convolutional Layer

32x32x3 image

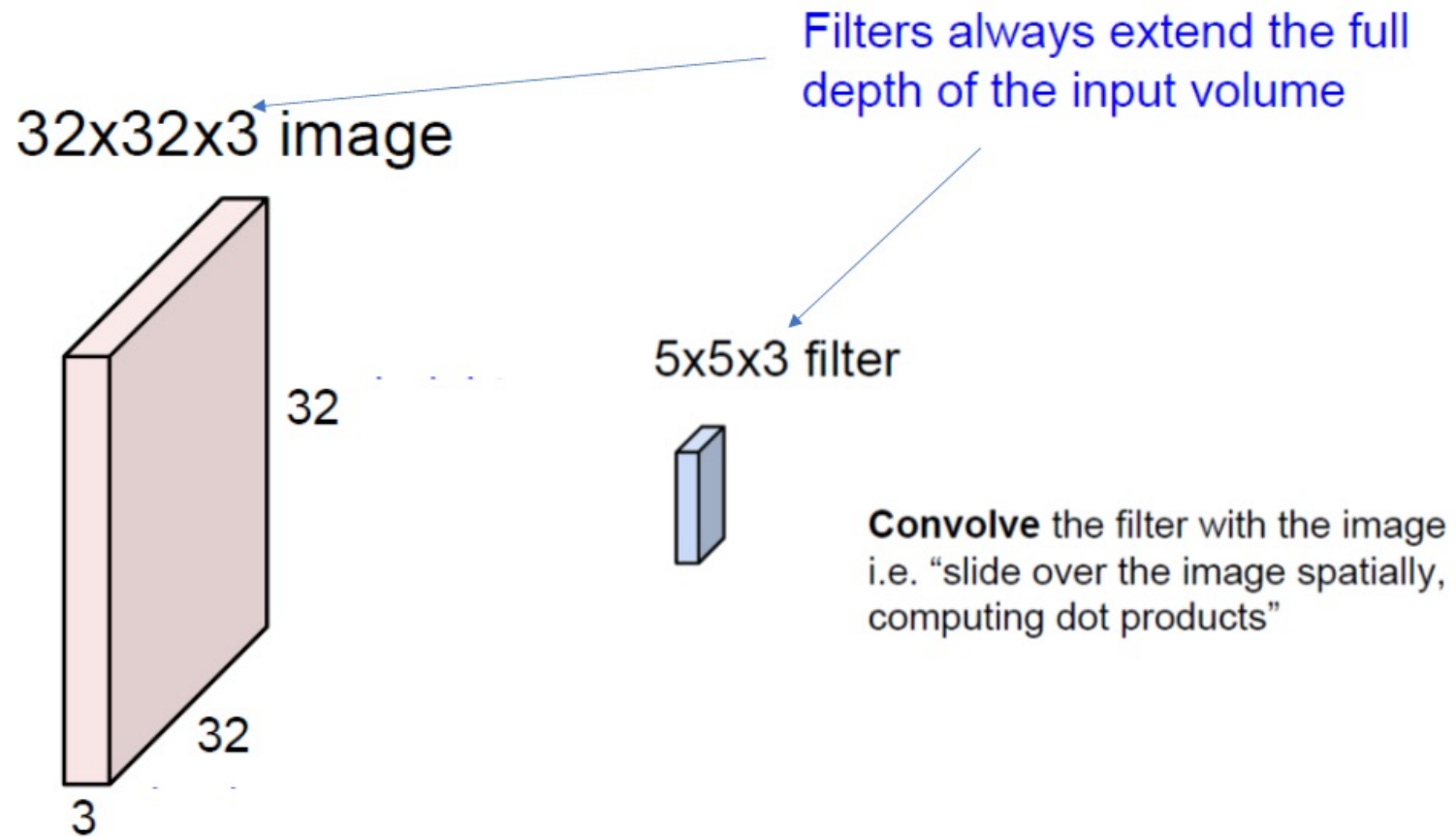


5x5x3 filter

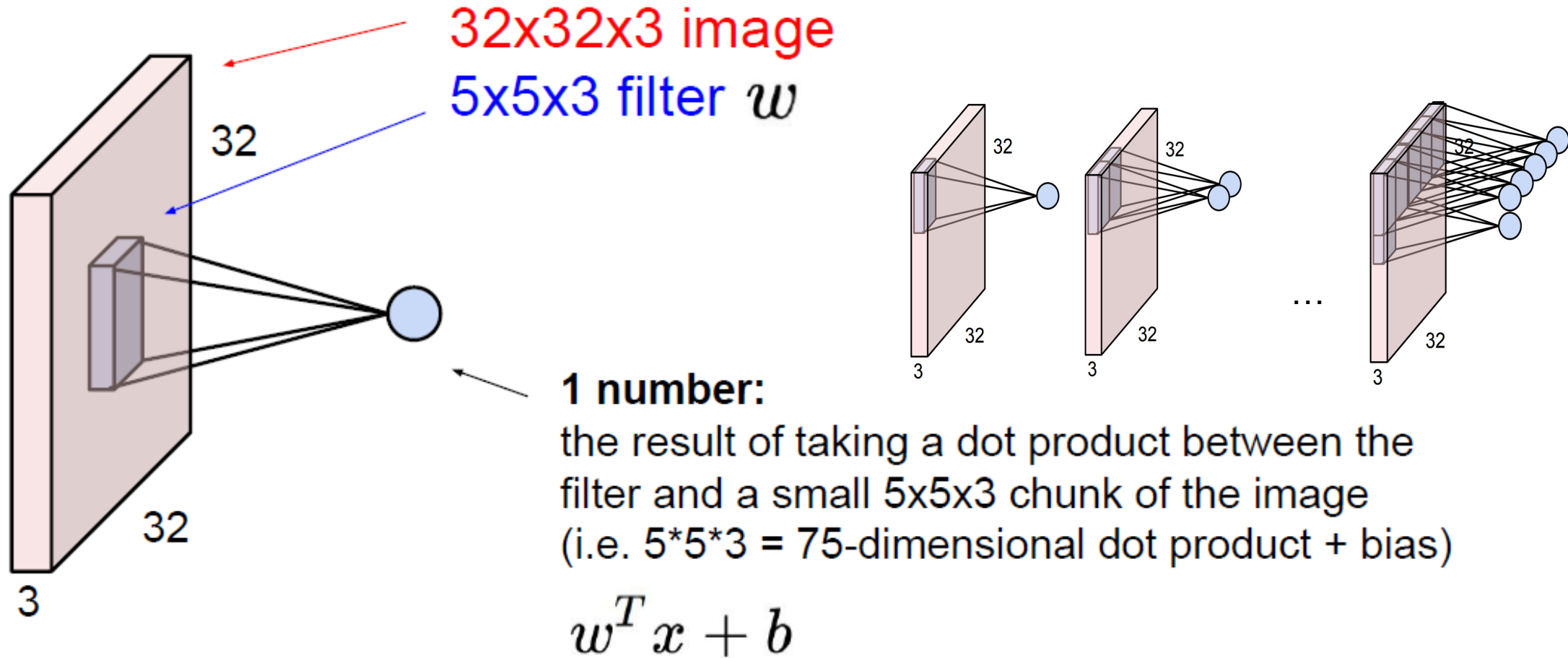


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

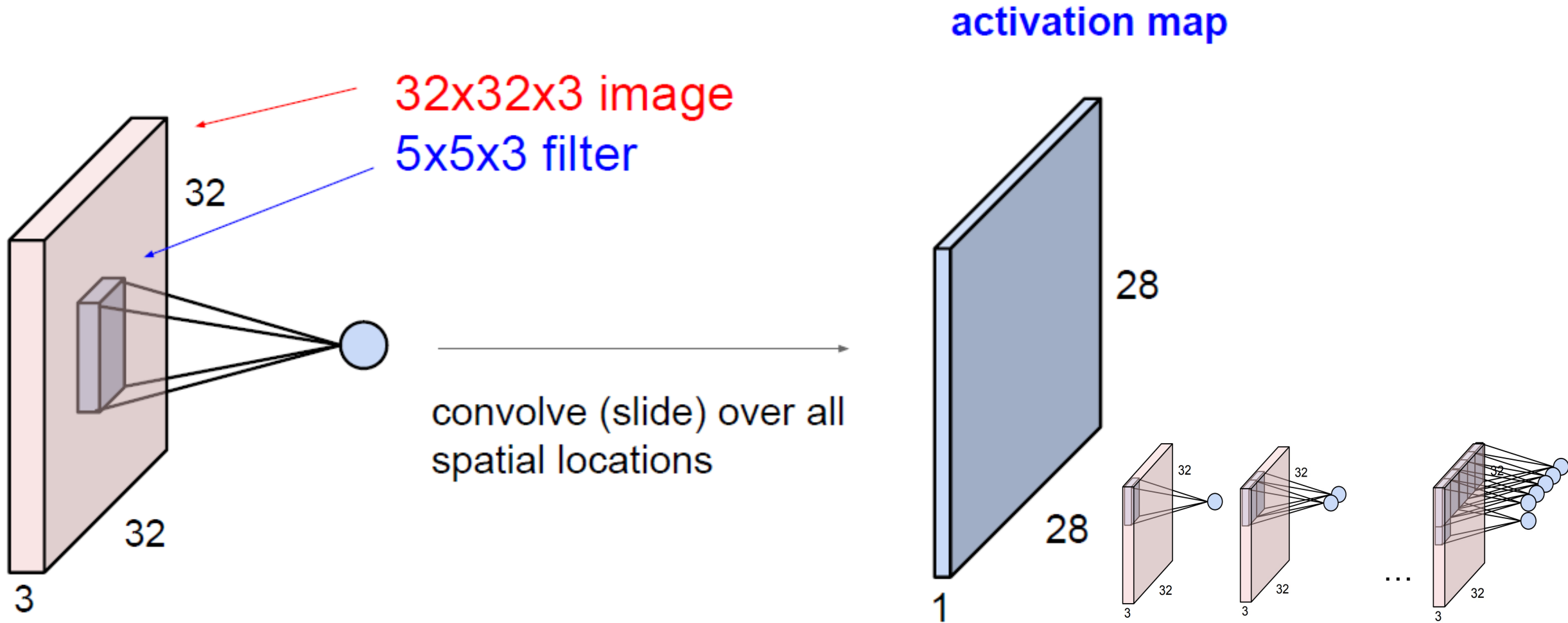
CNN: Convolutional Layer



CNN: Convolutional Layer



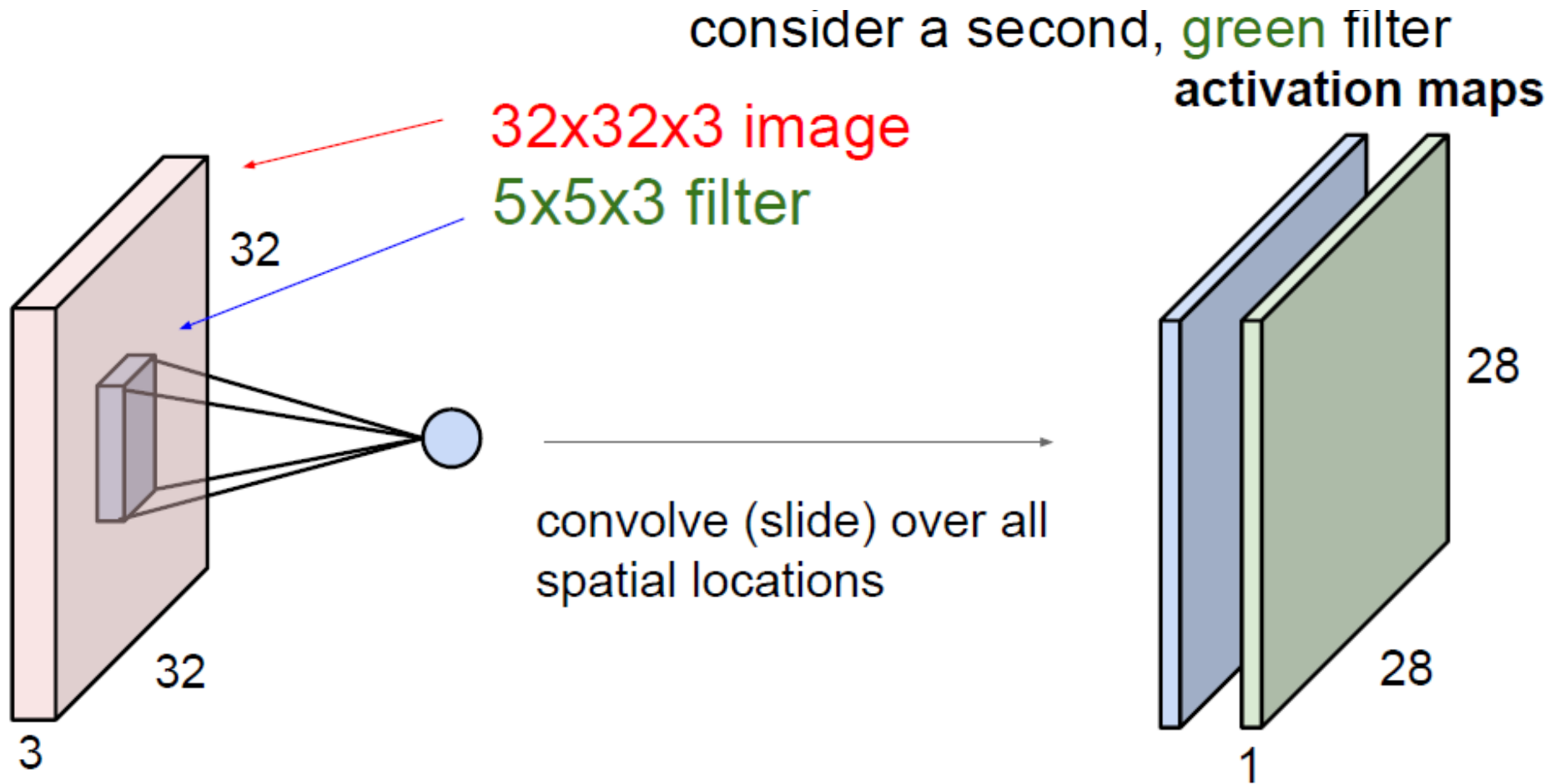
CNN: Convolutional Layer



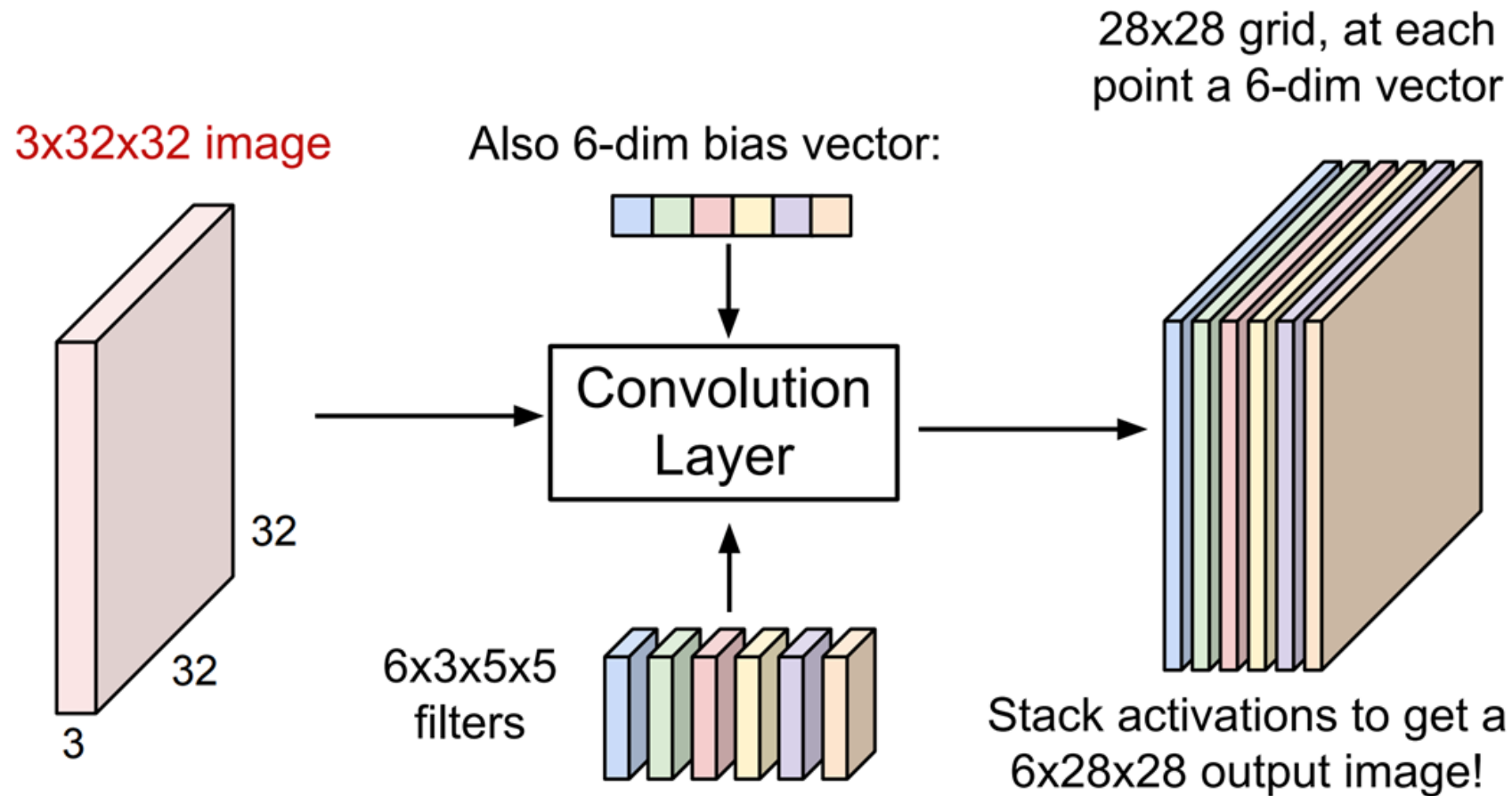
CNN: Convolutional Layer

- The output of the Conv layer can be interpreted as holding neurons arranged in a 3D volume.
- The Conv layer's parameters consist of a set of **learnable filters**. Every filter is small spatially (along width and height) but extends through the full depth of the input volume.
- During the forward pass, each filter is slid (convolved) across the width and height of the input volume, producing a 2-dimensional activation map of that filter.
- **Network will learn filters (via backpropagation) that activate (through the activation function) when they see some specific type of feature at some spatial position in the input.**

CNN: Convolutional Layer

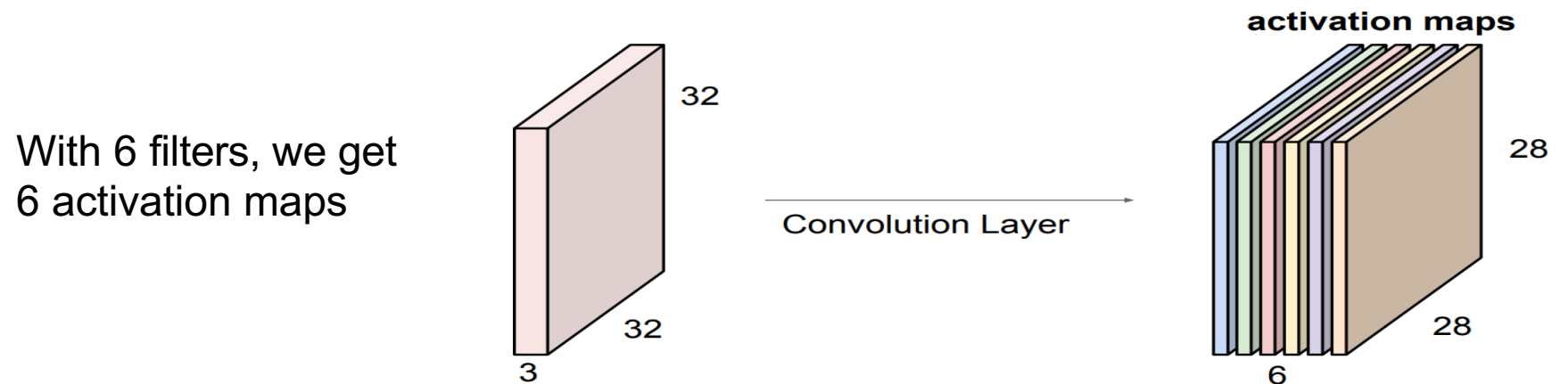


CNN: Convolutional Layer

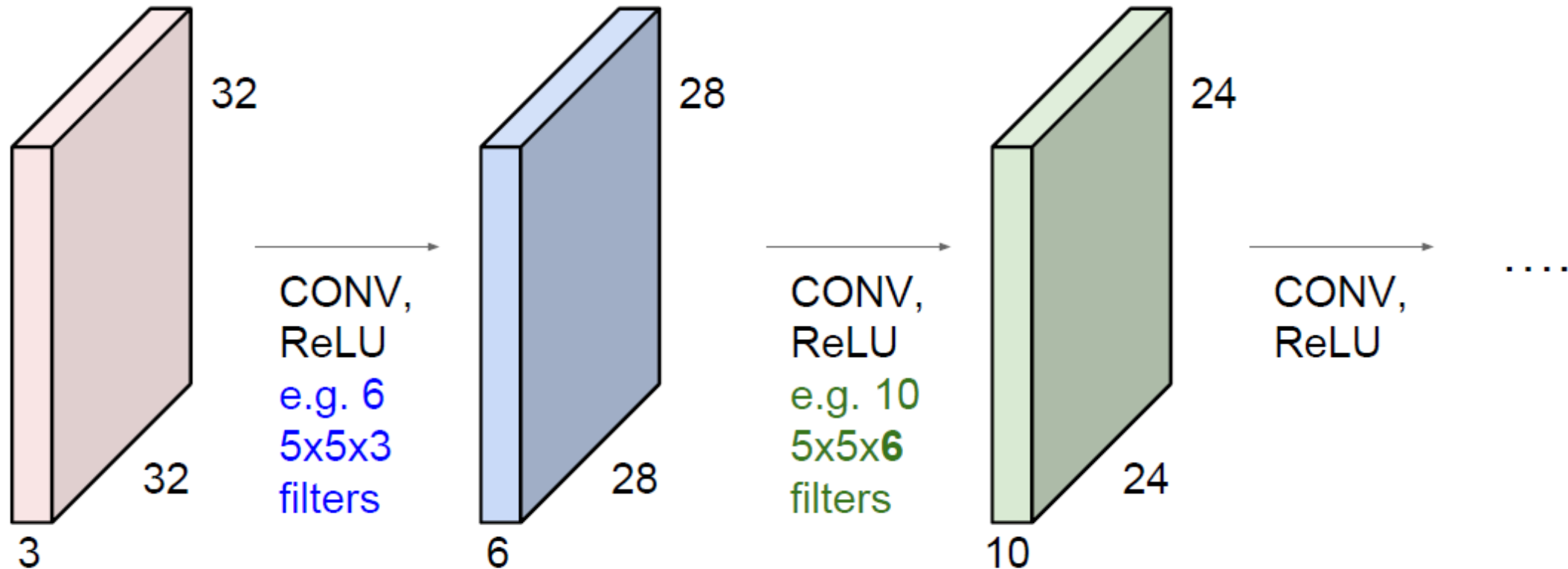


CNN: Convolutional Layer

- Stacking these activation maps for all filters along the depth dimension forms the full output volume
- Every entry in the output volume can thus also be interpreted as an output of a neuron that looks at only a small region in the input and ***shares parameters*** with neurons in the same activation map (since these numbers all result from applying the same filter)



Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions

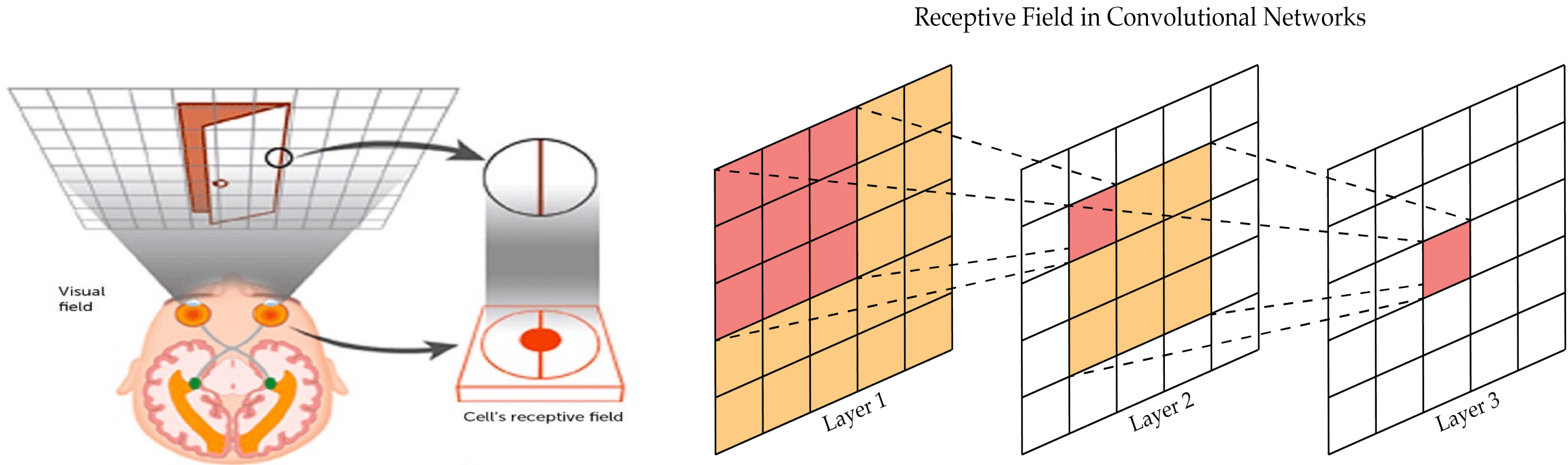


CNN: Convolutional Layer

Local Connectivity

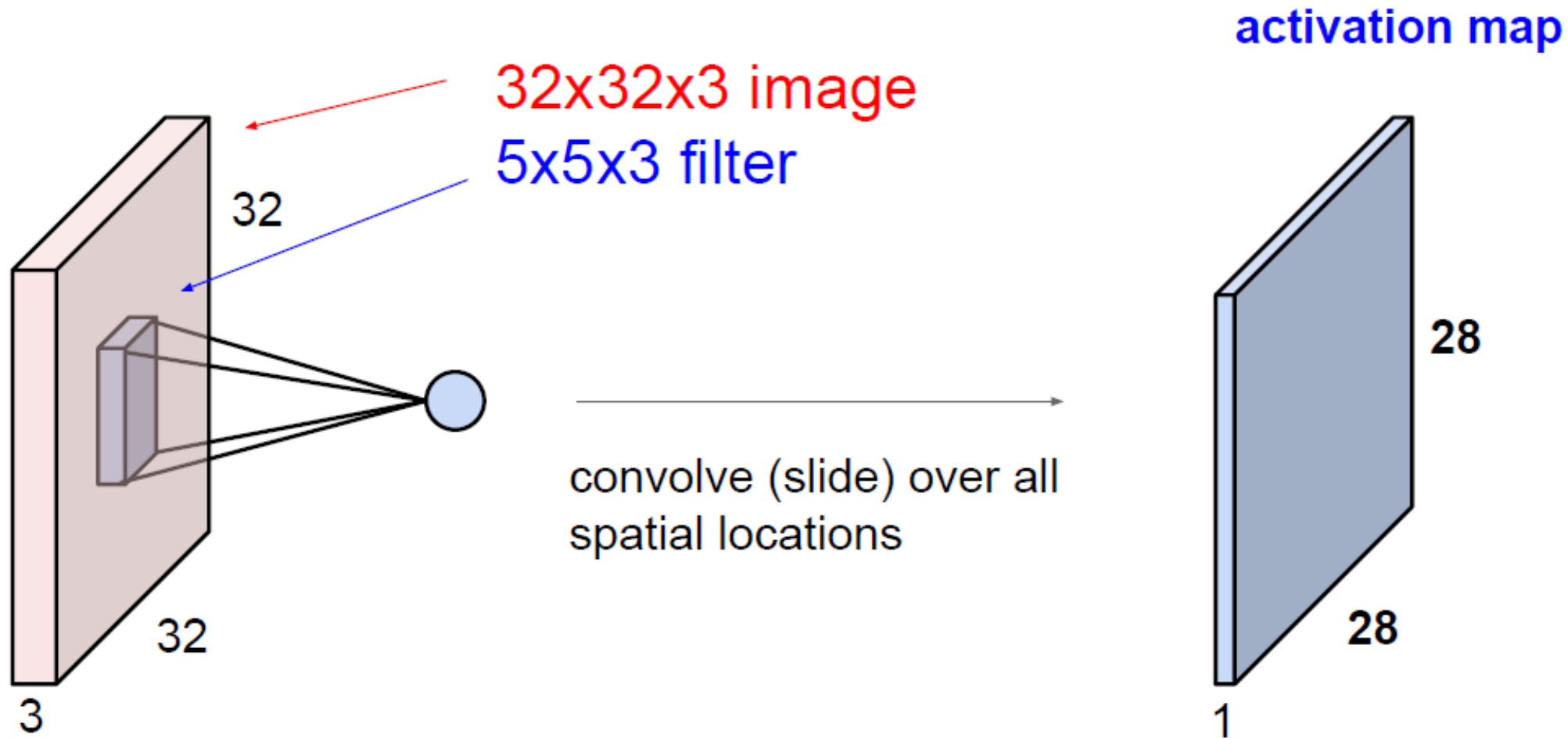
- As we have realized by now, it is impractical to use fully connected networks when dealing with high dimensional images/data
- Hence the concept of local connectivity: each neuron only connects to a local region of the input volume.
- The spatial extent of this connectivity is a concept called ***receptive field*** of the neuron.
- The extent of the connectivity along the depth axis is always equal to the depth of the input volume.
- The connections are local in space (along width and height), but always full along the entire depth of the input volume.

CNN: Receptive Field



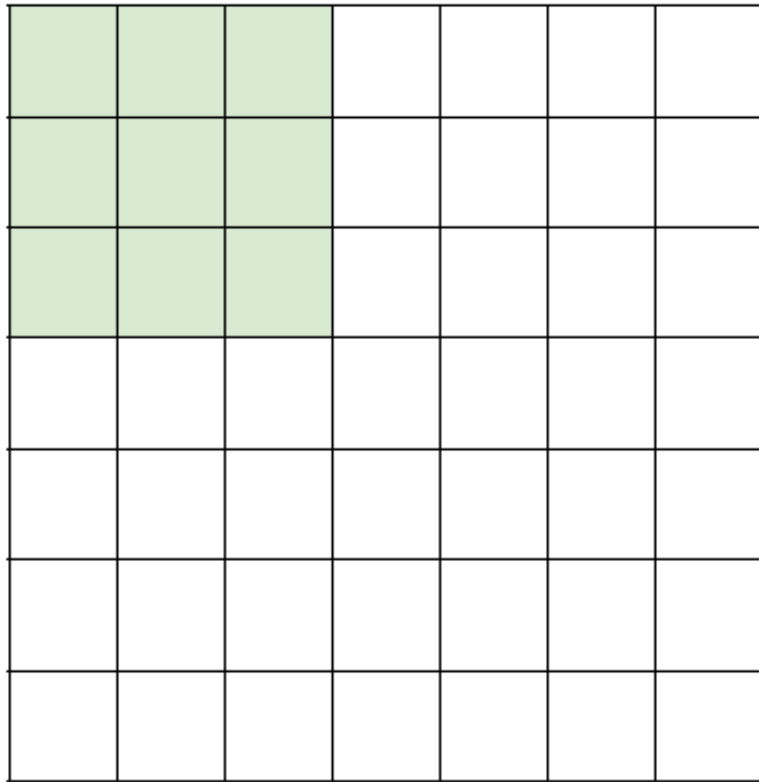
AI Summer. <https://theaisummer.com/receptive-field/>

A closer look at spatial dimensions



A closer look at spatial dimensions

7

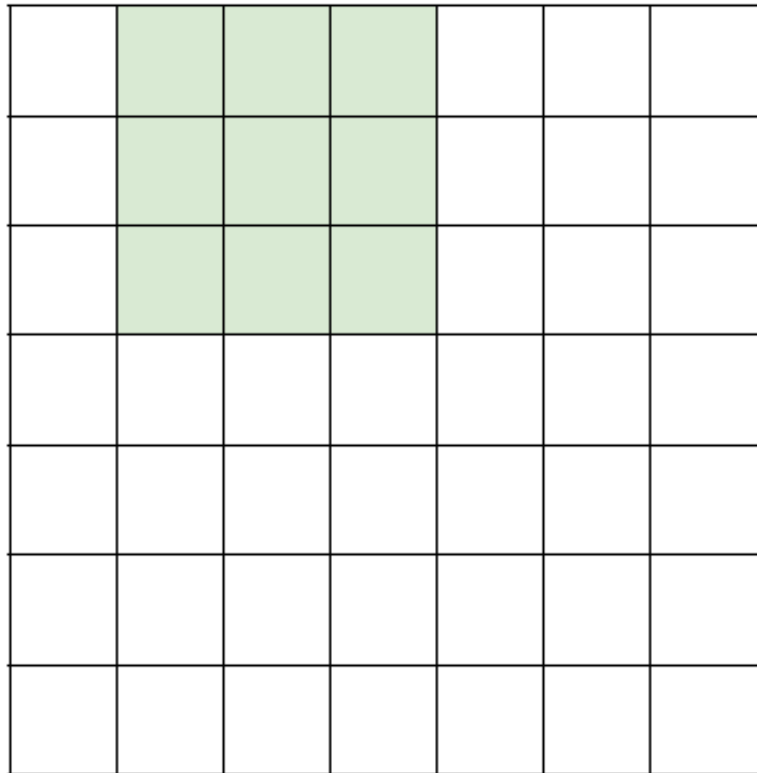


7x7 input (spatially)
assume 3x3 filter

7

A closer look at spatial dimensions

7

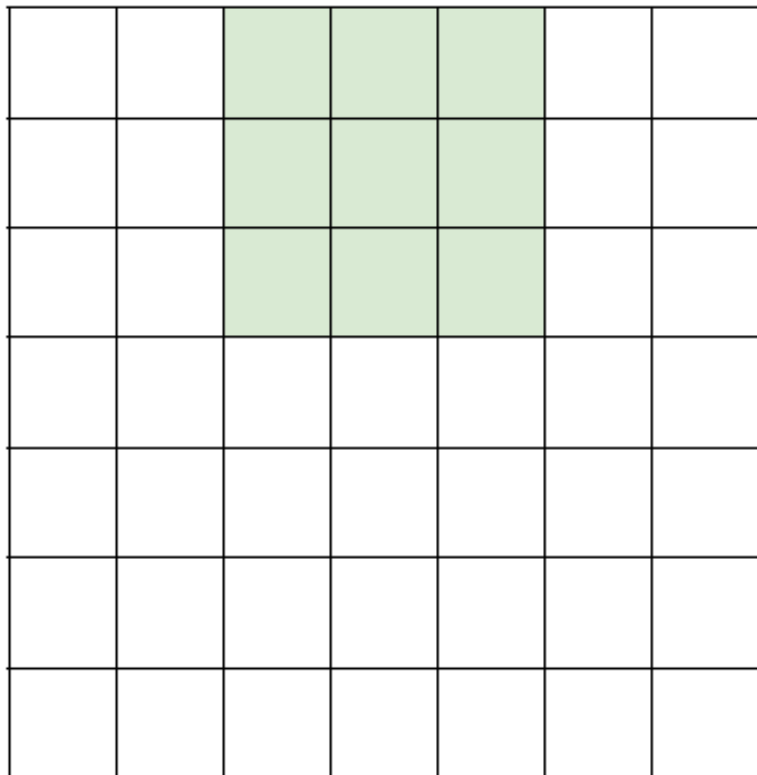


7x7 input (spatially)
assume 3x3 filter

7

A closer look at spatial dimensions

7

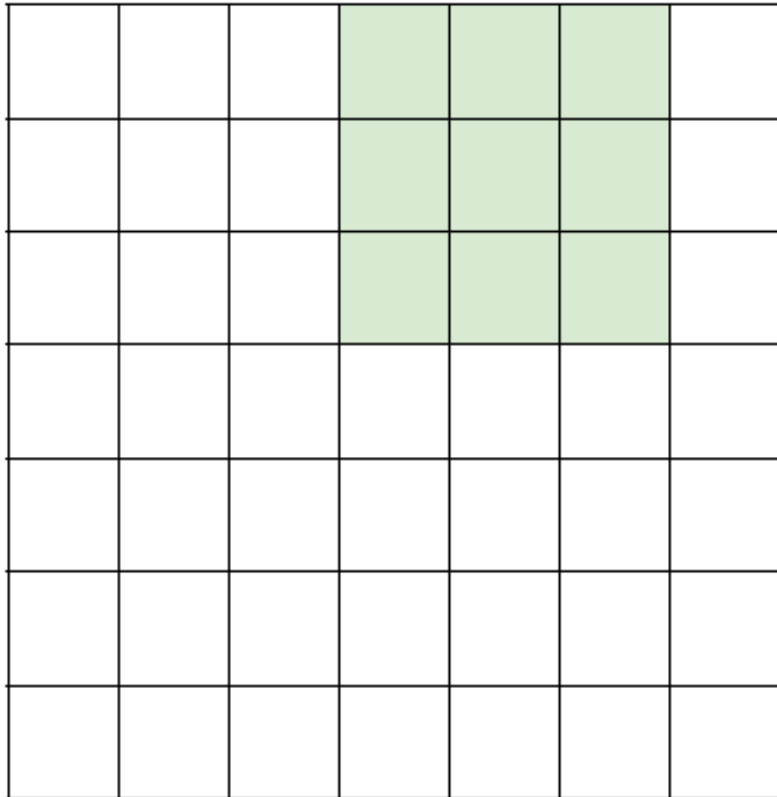


7x7 input (spatially)
assume 3x3 filter

7

A closer look at spatial dimensions

7

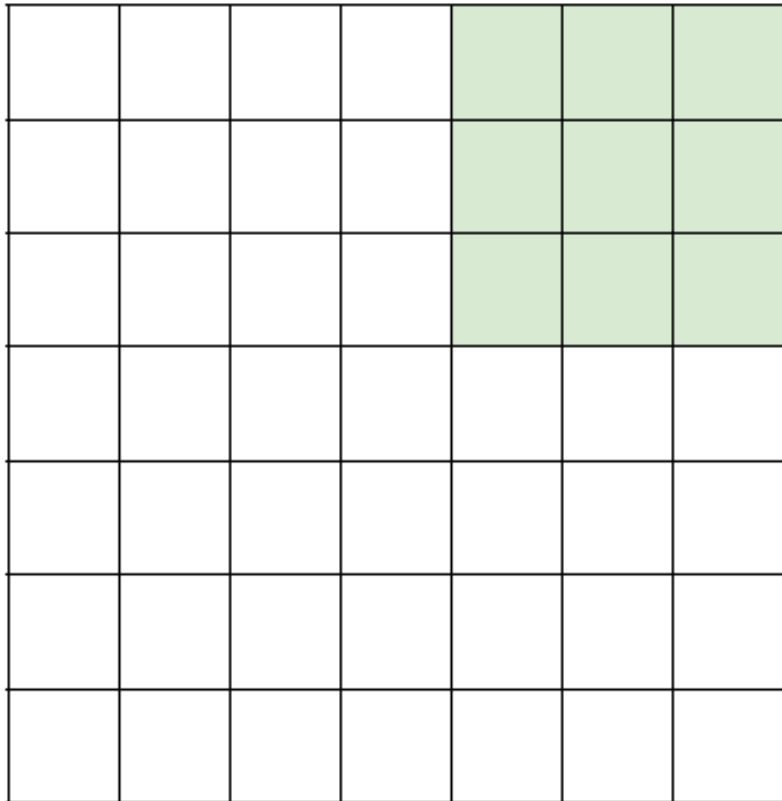


7x7 input (spatially)
assume 3x3 filter

7

A closer look at spatial dimensions

7



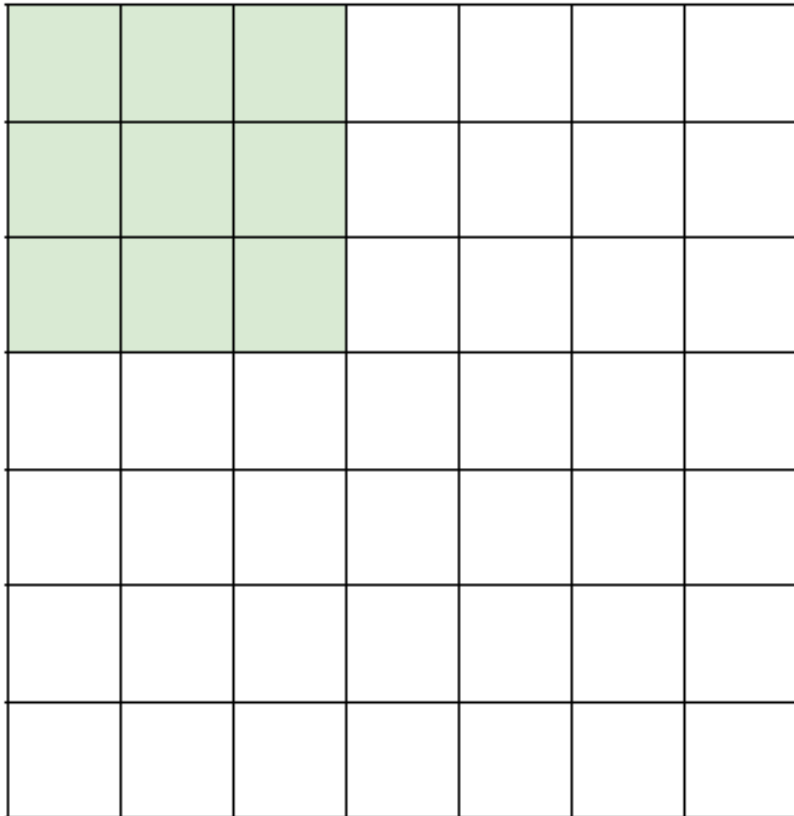
7x7 input (spatially)
assume 3x3 filter

=> **5x5 output**

7

A closer look at spatial dimensions

7

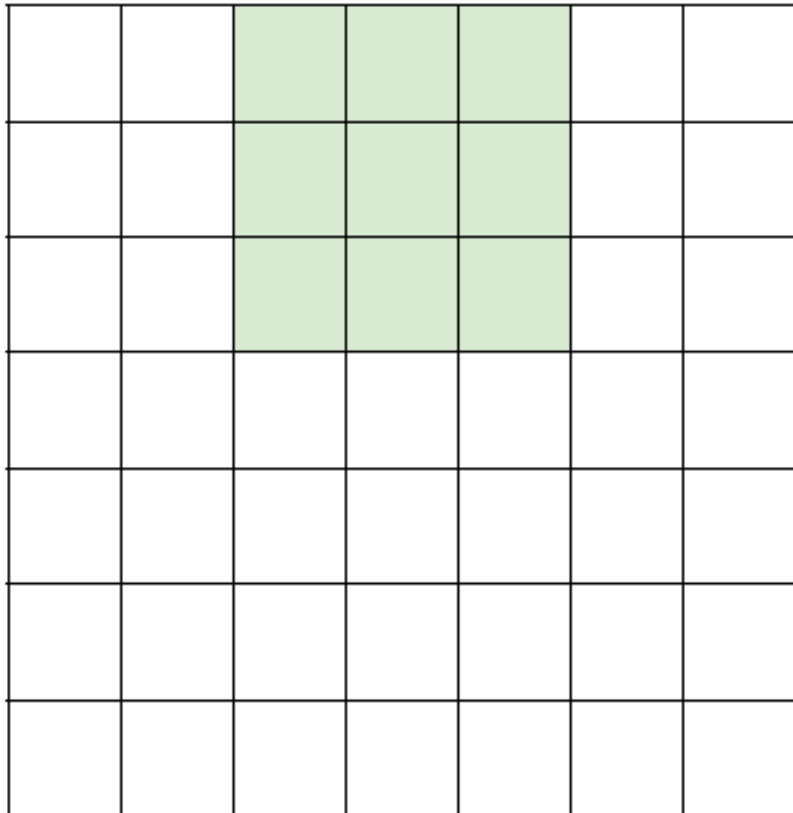


7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

7

A closer look at spatial dimensions

7

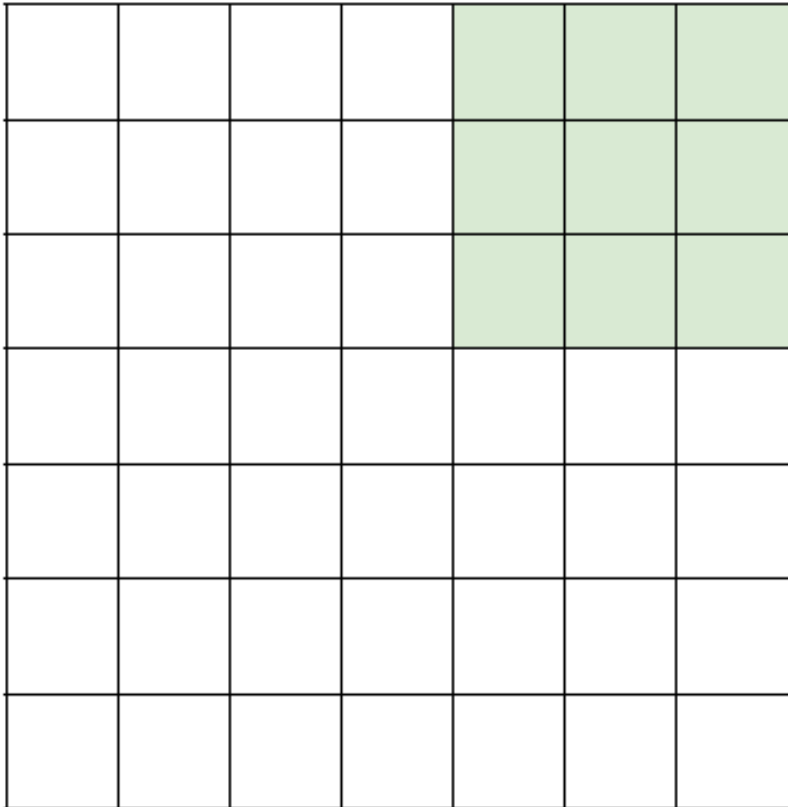


7

7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions

7

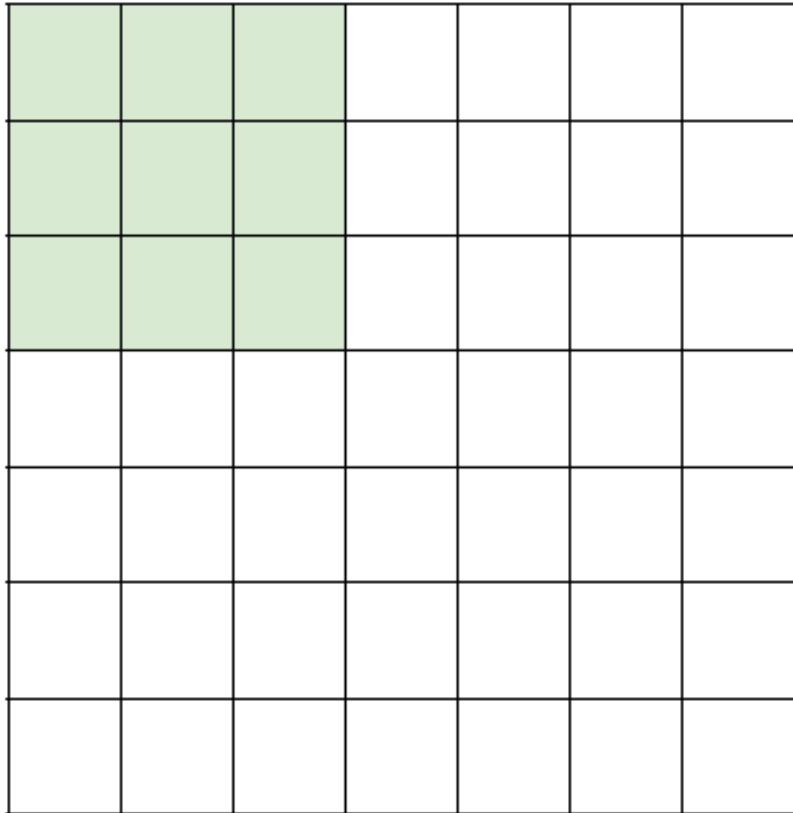


7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

7

A closer look at spatial dimensions

7

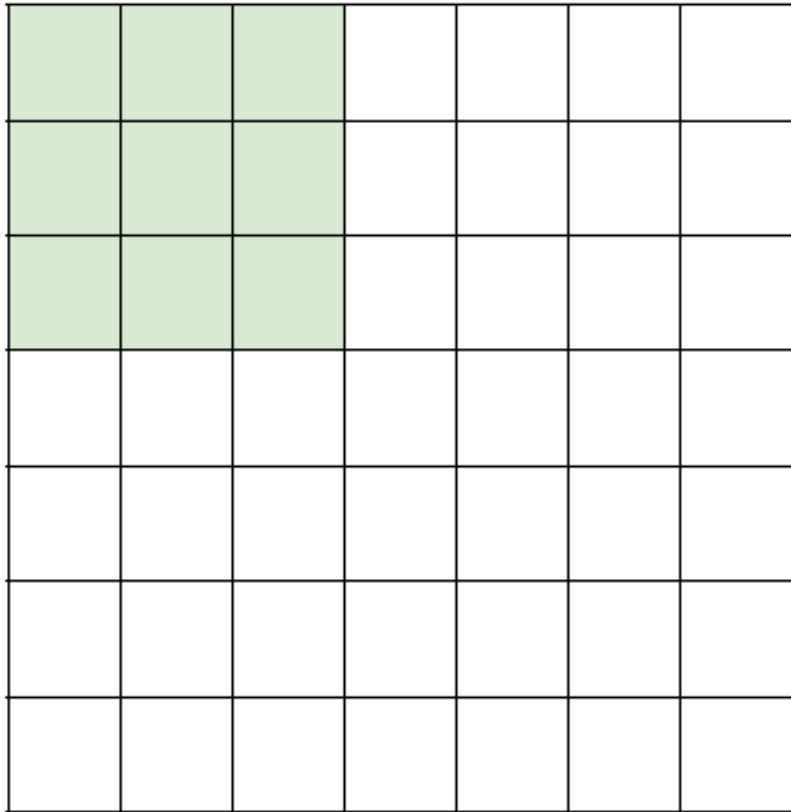


7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

7

A closer look at spatial dimensions

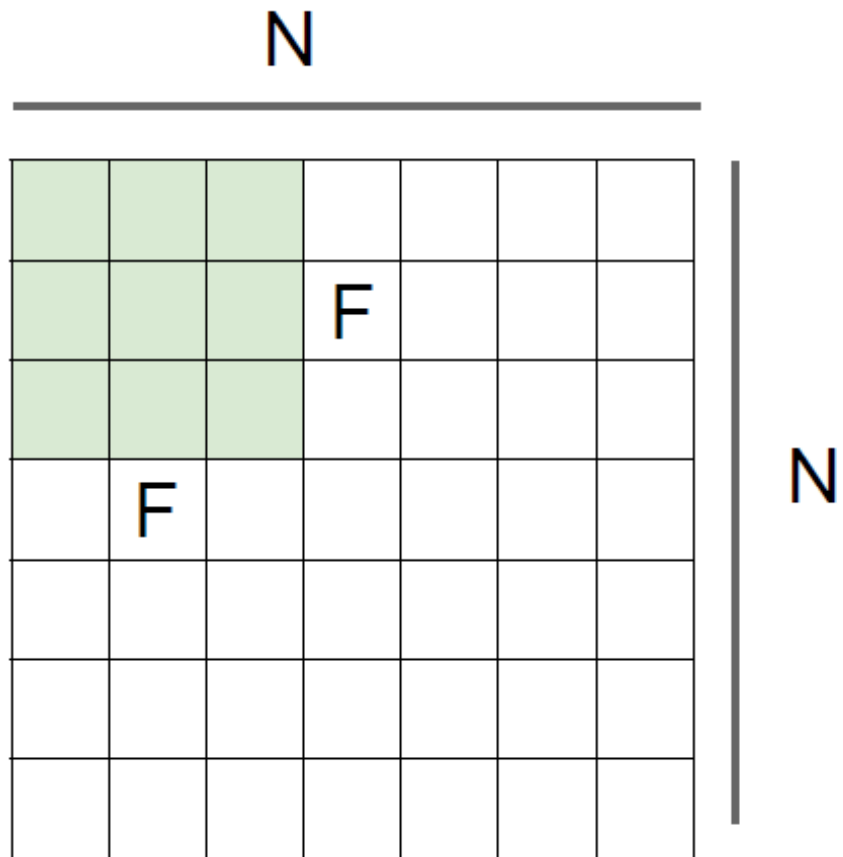
7



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

7

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.



Output size:
 $(N - F) / \text{stride} + 1$

e.g. $N = 7, F = 3$:

stride 1 $\Rightarrow (7 - 3) / 1 + 1 = 5$

stride 2 $\Rightarrow (7 - 3) / 2 + 1 = 3$

stride 3 $\Rightarrow (7 - 3) / 3 + 1 = 2.33 \therefore \backslash$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$$(N - F) / \text{stride} + 1$$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$$(N - F) / \text{stride} + 1$$

7x7 output!

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

e.g. $F = 3 \Rightarrow$ zero pad with 1

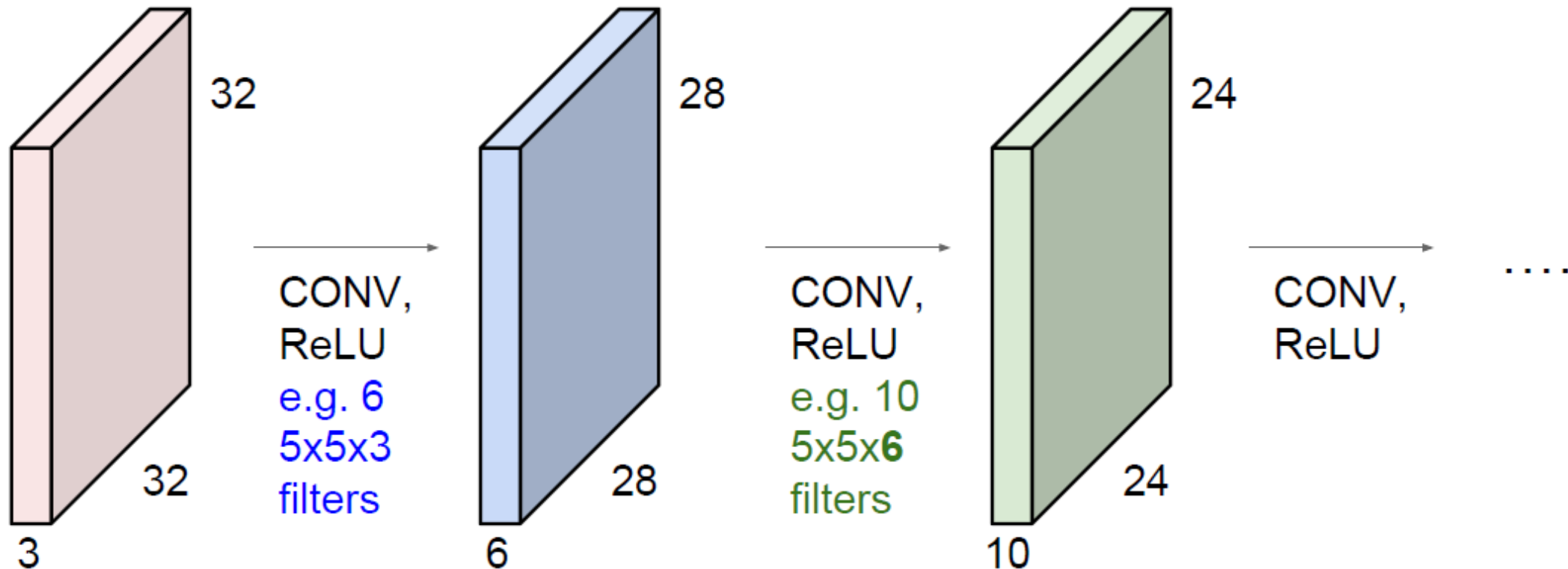
$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

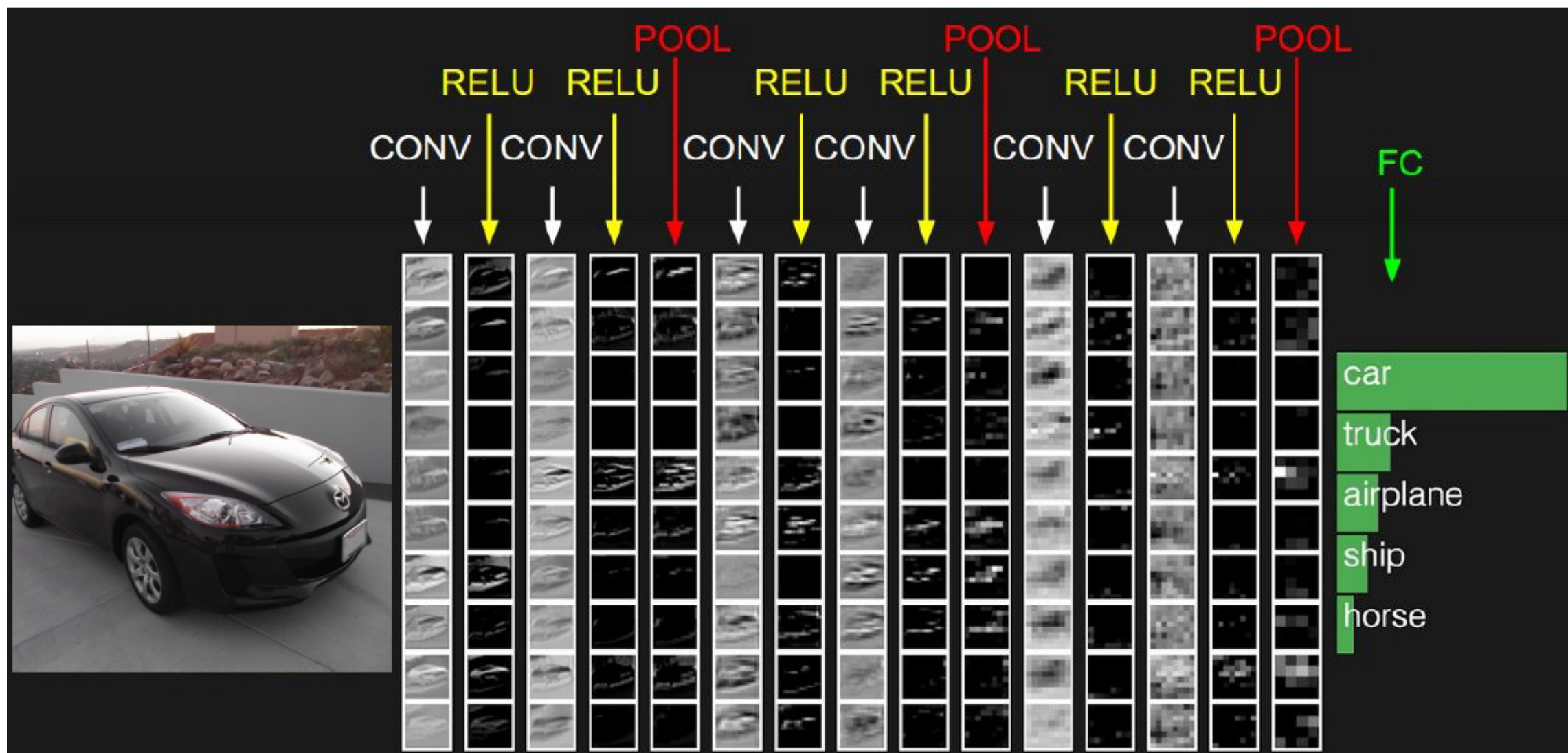
Other padding operations: replication padding, reflection padding ...

Remember back to...

E.g. 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 -> 28 -> 24 ...). Shrinking too fast is not good, doesn't work well.



two more layers to go: POOL/FC



J, K: Image dimensions

M, N: Kernel (Filter) dimensions

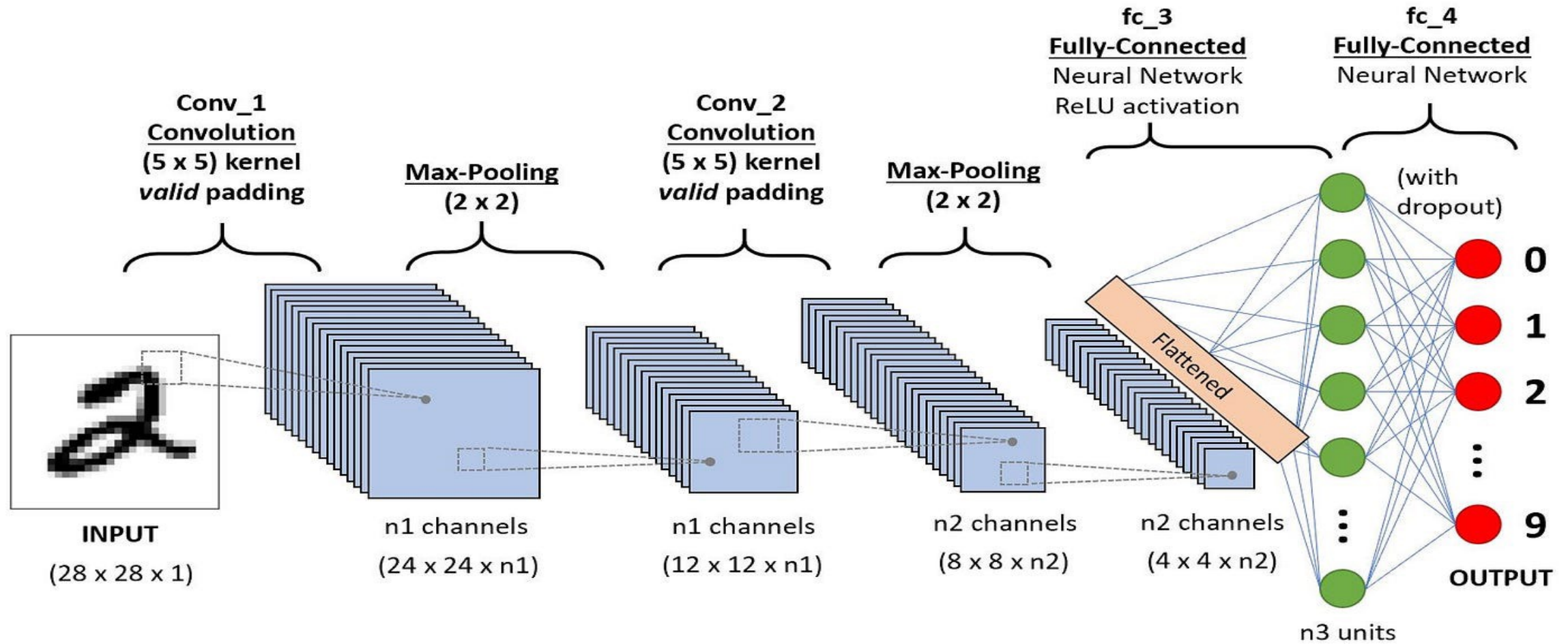
P: Padding size

s: stride

The size of the activation map is

$$(1 + (J + 2P - M) / s) \times (1 + (K + 2P - N) / s)$$

Let's understand numbers in CNN



Key takeaways

- CNN Basics
 - Specialized for image-related tasks.
 - Utilize convolutional and pooling layers.
- Continuous improvement in CNN architectures and heuristics (tips and tricks)
 - always check literature to find state-of-the-art methods
- Convolutional Filters
 - Detect features using filters/kernels.
- Training
 - Backpropagation, optimization, loss functions.

Acknowledgements

- Slides from
 - <http://www.analyticsvidhya.com/blog/2017/04/comparison-between-deep-learning-machine-learning/>
 - <https://towardsdatascience.com/convolutional-neural-networks-for-all-part-i-cdd282ee7947>
 - <https://www.slideshare.net/NirthikaRajendran/cnn-126271677>
 - <https://research.aimultiple.com/data-augmentation-techniques/>
 - <https://training.galaxyproject.org/training-material/topics/statistics/tutorials/CNN/slides-plain.html>
- Some material drawn from referenced and associated online sources...
 - Credit: Srivastava et al., Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 2014 https://colab.research.google.com/github/d2l-ai/d2l-en-colab/blob/master/chapter_multilayer-perceptrons/dropout.ipynb
 - Credit: Stanford CS231n course slides. Machine Learning Mastery: How to Normalize, Center, and Standardize Image Pixels in Keras
 - Source: LearnOpenCV: Batch Normalization in Deep Networks. <https://learnopencv.com/batch-normalization-in-deep-networks/>
 - Source: Towards Data Science. Various Optimization Algorithms For Training Neural Network <https://towardsdatascience.com/optimizers-for-training-neural-network-59450d71caf6>